

Short Paper

A Straightforward Approach to Construct “Lightweight” Token Graphs from Transformers

João Pimentel^{*1,2}, Joana Amorim^{1,2}, Frank Rudzicz^{1,2}

¹Faculty of Computer Science, Dalhousie University, Halifax, Nova Scotia, Canada
joao.pimentel@dal.ca

²Vector Institute for Artificial Intelligence, Toronto, Ontario, Canada

Even though Transformers are the standard to extract information from text, they have considerable limitations that make them sub-optimal for certain tasks. Graph neural networks (GNNs) are an alternative that mitigate some of these limitations, with recent work focusing on combining information from (PLMs) with graph structures. However, most existing methods use fixed-size sliding windows to construct graphs, ignoring long-distance relationships between words. We propose a memory-efficient method to construct token graphs that exploits semantic and relational information inside Transformers via their attention coefficients, mitigating the limitations of using sliding windows. Additionally, each graph can fully encode documents longer than the Transformer’s context window via a chunk-and-stride mechanism, while also reducing memory usage. Our method surpasses the performance of sliding windows when both approaches are compared under the same model architecture, particularly in longer documents, and reaches statistically identical performance in shorter ones. It also surpasses GNN-based state-of-the-art techniques. When compared with fine-tuning, our method reaches marginally lower performance in documents that fit inside the pre-trained model’s context window. However, surpasses it in documents that do not fit inside these windows, which comes as a result of fully encoding each document using our chunk-and-stride mechanism.

1. Introduction

The amount of publicly available unstructured text data is consistently increasing, leading to a need for improved information extraction (Taha et al. 2024). Several popular approaches in natural language processing (NLP), such as bidirectional encoder representations from transformers (BERT) (Devlin et al. 2019) or generative pre-training

* Corresponding author.

transformer (GPT)-2 (Radford et al. 2019), use the Transformer architecture. However, training or fine-tuning a Transformer does not guarantee improved performance. In fact, depending on the data, this architecture may be entirely unusable (e.g., when all documents have many more tokens than the pre-trained language model’s [PLM’s] context window). This comes as a result of Transformers being limited by maximum sequence lengths, which originate from their high computational requirements (Burdisso et al. 2023).

Graph neural networks (GNNs) may be a potential solution to mitigate some of the limitations from which Transformers suffer, while also achieving competitive results (e.g., in sentiment polarity and depression screening tasks) (Burdisso et al. 2023; Zhang et al. 2022). Even though some methods use a single, corpus-wide heterogeneous graph composed of both document and word nodes from which to learn (Yao, Mao, and Luo 2019; Burdisso et al. 2023), others have shifted towards inductively representing each document as an independent graph of words (Nikolentzos, Tixier, and Vazirgiannis 2020; Zhang et al. 2020). By doing so, these methods consider context-based word relationships (Zhang et al. 2020) alongside document structure (Huang et al. 2019), while requiring less memory (Huang et al. 2019) when compared with having a single heterogeneous graph.

Some methods that represent each document as its own homogeneous graph of words do so by constructing graphs based on co-occurring words within fixed-size sliding windows (e.g., message passing attention network for document understanding (MPAD) [Nikolentzos, Tixier, and Vazirgiannis 2020] or a text classification method for inductive word representations via GNNs (TextING) [Zhang et al. 2020]). However, these windows are often narrow, meaning that distant word interactions and important syntactic dependencies may be ignored (Wang et al. 2024; Jin et al. 2022). Additionally, most graph-based methods bind words to a single representation by using older word embeddings (e.g., word-to-vector (word2vec) [Mikolov et al. 2013] or global vectors (GloVe) [Pennington, Socher, and Manning 2014]), ignoring potential contextual variation in meaning (Wang et al. 2024). To tackle these problems, some researchers have focused on empowering co-occurrence graphs with extra information, showing benefits in considering broader context windows, as well as positional embeddings (Jin et al. 2022). Others have tried to combine aspects of pre-trained models with graphs. While some simply replaced traditional embeddings with context-aware ones (Yang and Cui 2021), others entirely rethought graph construction (Zhang et al. 2022). The latter have given us promising indications that the data within some PLMs can be used to derive more informative aspects that relate the words in a text than close-distance co-occurrences (Zhang et al. 2022). Nonetheless, how to best exploit different PLMs and their inner information remains an open problem. Our work follows this gap by exploring a new, straightforward way to assemble word graphs using PLMs, where attention coefficients are key to establish the set of edges and, consequently, nodes, too.

1.1 Contributions

We propose a data-driven method that uses both the attention coefficients and token embeddings of a PLM as the basis for graph construction. This method differs from other work that combined PLMs with graphs (Yang and Cui 2021; Zhang et al. 2022) by solely relying on attention coefficients to construct the edge set. By doing so, the relationships of each word are not limited to a pre-defined distance, with each node having its own “window” of variable size—possibly considering more distant word interactions and syntactic dependencies (Htut et al. 2019; Ravishankar et al. 2021). Even

though we rely on Transformers for construction, we are not limited by maximum sequence lengths nor very high computational requirements. This is a direct result of chunking long texts and filtering unimportant edges and nodes below a pre-defined threshold. Additionally, the graphs are constructed through simple inference, meaning we do not require extra training for construction, unlike other methods.

We demonstrate that this combination of meaningful information from PLMs with structural learning from GNNs achieves competitive results when compared to graph-based state-of-the-art (SotA) techniques, as well as sparser, but more informative, word graphs for subsequent analysis. The main contributions of this work are:

- A novel, model-agnostic method to construct token graphs using PLMs, as long as these have attention coefficients.
- A detailed comparison between different configurations of the proposed method with many graph-based SotA approaches.
- A comparative analysis between using our method with using sliding windows to construct document-level graphs, as well as with fine-tuning the PLM, instead.
- An assessment of the memory requirements of models and data representation comparing our method with sliding windows and fine-tuning.

2. Related Work

This section provides detailed insights about traditional word co-occurrence-based approaches, as well as more recent techniques that either extend the former by leveraging other text properties or that rely on the knowledge from PLMs instead of co-occurrences. Given our approach, we focus on work that builds one graph per document.

2.1 Exclusively Learning from Word Co-occurrences

Text-level GNN (Huang et al. 2019) pioneered the departure from having a single heterogeneous graph by representing each document as its own sub-graph of a corpus-wide word graph. Its learning stage was done via a message passing mechanism (Gilmer et al. 2017) that helped inject corpus-level information into nodes and edges. Although some dependency between documents and the corpus is eliminated, text-level GNN remained transductive—all tokens must be in the training data to be properly represented.

MPAD (Nikolentzos, Tixier, and Vazirgiannis 2020) and TextING (Zhang et al. 2020) were proposed as solutions to this dependency problem by creating an independent graph for each document. MPAD also introduced document-wide properties via “master nodes” (i.e., a central node connected to all other nodes in each graph), and extracted hierarchical information through intermediate pooling after each gated GNN (Li et al. 2016) layer. In contrast, TextING was a more streamlined solution, exclusively focusing on word co-occurrences for graph construction and adding a soft-attention mechanism for global aggregation instead of intermediate pooling layers.

All three methods surpassed previous heterogeneous approaches, such as text graph convolutional network (TextGCN) (Yao, Mao, and Luo 2019), while also achieving

a considerable reduction in memory usage and model complexity. Only TextING surpassed text-level GNN (by an average of $\approx 0.71\%$ accuracy), accentuating the benefits of focusing on in-document word relationships, as well as using a simpler model architecture. However, by solely relying on co-occurrences to construct the set of edges, ignoring the flow of text, and representing nodes with GloVe embeddings, TextING missed important contextual information for each word.

2.2 Extending the Co-occurrences

Several attempts have been made to extend co-occurrence graphs to account for other information. For example, multi-feature hierarchical graph attention network (MH-GAT) (Jin et al. 2022) combined syntactic dependencies and relative positional information with word co-occurrences. Part-of-speech and positional embeddings were combined with corpus-wide co-occurrence-based word embeddings that were then fed to a bi-directional long short-term memory and a graph attention network (GAT) (Veličković et al. 2018) to generate more informative embeddings. Using these as node features, MH-GAT built hierarchical syntactic dependency and co-occurrence graphs for each document to train a hierarchical GAT.

Zhao et al. (2024) proposed that co-occurrences and sentence-level syntactic dependencies could be combined as a way to build edges. Two graphs were constructed for each document and then fused via pooling over the adjacency matrices following an along-edge attention mechanism to weigh the importance of distinct edges. Subsequently, noisy connections were dynamically eliminated. Although experiments showed that co-occurrences have the highest effect on performance, fusing them with extra information positively affected performance. Experiments also showed that not dynamically pruning edges significantly decreased performance—indicating the importance of filtering unimportant information prior to learning.

Other work tried to learn from hypergraphs, which are graph extensions in which edges are not limited to connect just two nodes (Ding et al. 2020). Ding et al. (2020) proposed a method to represent each document as a hypergraph in hypergraph attention network (HyperGAT). On one hand, nodes depict words, being represented by one-hot encoding vectors or traditional word embeddings. On the other hand, hyperedges model both (i) co-occurrences inside individual sentences to capture document structure (Yang et al. 2016) and (ii) the top N most likely latent topics that correlate with words in each document (Linmei et al. 2019), based on latent Dirichlet allocation (Blei, Ng, and Jordan 2002). Node embeddings were updated using two aggregation operations and a dual attention mechanism, so that the most informative combinations of nodes and hyperedges can be accounted for during the updates. While HyperGAT’s average performance was $\approx 1.25\%$ higher than that of text-level GNN, the performance change was minimal from adding topic-based information—indicating that most of the learning was done via co-occurrences.

Jin et al. (2024) extended the previous idea with multi-element hypergraph gated attention network (MeHGAT) by looking at different document properties. Each text was represented by positional information, alongside sentence- and document-level properties obtained through hierarchical average pooling. Like the previous method, words were represented as nodes, using similar features. Sentence-level hyperedges connected all words within the same sentence, while document-level ones connected all nodes in the document, with the properties of both hyperedges being calculated using node features. By contrast, positional hyperedges connected words in the same absolute position across different sentences via positional embeddings (Vaswani et al.

2017). Gated attention was then used to filter unimportant information during learning. MeHGAT was, on average, $\approx 1.18\%$ more accurate than TextING and $\approx 1.26\%$ more accurate than HyperGAT, with ablation studies showing that dropping positional hyperedges had the most negative impact on performance. Nevertheless, adding both sentence- and document-level structural features effectively enhanced performance.

The previous methods highlight the importance of combining relevant information from different paradigms, particularly semantic properties, syntactic dependencies, and positional information to achieve more accurate results. Even though pruning is ignored by most methods, it also proves to be an important post-processing stage of graph construction. However, by not learning from context-aware embeddings, these techniques may miss crucial details in the text, while also being limited by pre-existing vocabularies (Donabauer and Kruschwitz 2025). As a result, recent works have started to shift towards using richer embeddings, specifically embeddings from PLMs that account for the context in which words appear, and tokenization methods capable of dealing with unknown words using sub-word segments.

2.3 Combining Co-occurrences with PLMs

BERT-enhanced text graph neural network (BEGNN) (Yang and Cui 2021) combined BERT lexical features with structural information. BERT was used to tokenize each text and for semantic feature extraction. The resulting tokens were used to build undirected co-occurrence graphs based on sliding windows of size three, that were passed through a gated GNN (Li et al. 2016) to obtain structural features. Lexical and structural feature sets were aggregated through a mutually conditioned attention-mechanism, i.e., semantic features were input to the structural self-attention block and vice-versa. Experimentally, BEGNN consistently outperformed TextING by an average of $\approx 4.13\%$ and TextGCN by $\approx 6.83\%$ in varied topic classification tasks.

Donabauer and Kruschwitz (2025) introduced a similar yet simpler technique focused on short text classification. Each text was represented as an independent token-level graph in which nodes and their embeddings were entirely produced by a PLM. Edges were then constructed by linking all nodes inside a fixed-size sliding window in a fully connected fashion. Learning was accomplished via stacked GAT layers, with document-level embeddings obtained through average pooling. When evaluated using BERT as its basis, this method surpassed TextING, as well as fine-tuned versions of BERT using only 20 documents per class for training and validation splits. Results also showed that this technique performed better on data sets where sentences are longer and richer in semantic content, in contrast to texts with limited context.

Li et al. (2025) presented a method that constructed two distinct graphs to capture hierarchical contextual information: CoGraphNet. For each document, a word- and a sentence-level graph were constructed, where nodes represented words in the former and sentences in the latter. BERT was used to tokenize all documents, as well as to obtain token- and sentence-level embeddings. Edge weights were calculated through combining cosine similarity (Liu et al. 2004) with positional bias, i.e., co-occurrence counts were adjusted based on relative positional information. A hierarchical attention network (Yang et al. 2016; Wang and Wan 2019) alongside the SwiGLU activation function (Han and Zhang 2022; Narang et al. 2021) were then used for learning. CoGraphNet was $\approx 4.06\%$ more accurate than TextING on varied text classification tasks, indicating the benefits of the added graph components. Furthermore, extra experiments showed that combining both graphs consistently resulted in the best performance—once more,

showing the importance of relying on information that is not attainable using co-occurrences alone.

Even though these techniques extended co-occurrences in order to reduce the effects of some well-known limitations, the inherent structure of the constructed graphs was still limited by narrow, fixed-size sliding windows with all their subsequent problems. Consequently, edges may not only miss relevant connections, but also ignore how not all words relate to others equally.

2.4 Replacing Co-occurrences with the Information Within PLMs

Being aware of the intrinsic problems from relying on fixed-size sliding windows, researchers have started to shift from co-occurrence graphs (and their extensions) towards re-designing graph construction completely. One example was BERT-based hierarchical GAT (BHGA_{AttN}) (Zhang et al. 2022). Graphs exploited the hierarchical relationships from intermediate BERT layers, which were then learned by a multi-level hierarchical GAT. Semantic features for a predetermined number of sub-sentences were extracted from intermediate BERT layers via average pooling, while whole-sentence embeddings were randomly initialized. The GAT then considered the dependency order between whole- and sub-sentences, and updated the whole-sentence representations. Experimental results from sentiment analysis tasks suggested that the hierarchical structure within sentences contained more information than close-distance word co-occurrences, and that certain PLMs may be more adequate than others for downstream structural learning.

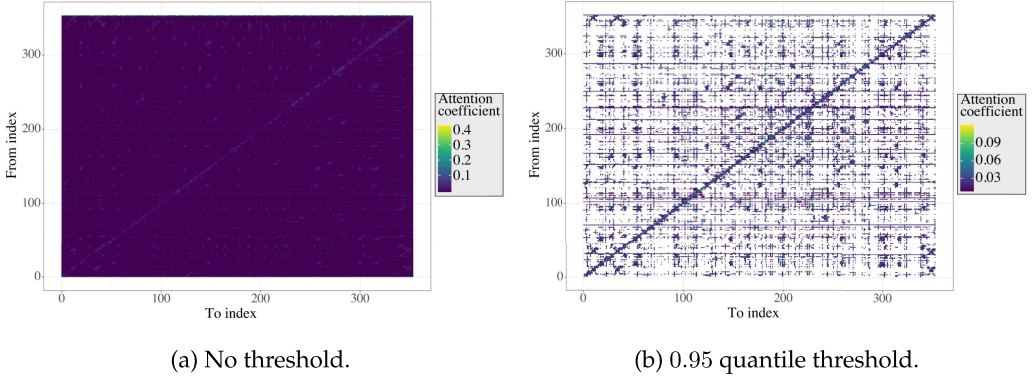
By contrast, Alikasıfoğlu, Aras, and Koc (2024) proposed visibility pooler (VISPool), which constructed vector visibility graphs (VVGs) (Ren and Jin 2019; Lacasa et al. 2008) using Transformers. Because token embeddings from Transformers can be seen as multivariate time-series with inherent sequential properties (which VVGs were designed to capture and map to nodes) these graphs completely preserve sequential and structural information. After all graphs were constructed, learning was accomplished using graph convolutional network (GCN) layers (Kipf and Welling 2017). Experiments on varied tasks using two distinct pre-trained models reiterated the previous insights that performance is affected by the choice of PLM. Furthermore, combining GNNs and PLMs can result in improved performance by learning structural information for which linear layers fail to account.

Despite the clear benefits that both BHGA_{AttN} and VISPool possess, both methods are considerably complex. For example, BHGA_{AttN} requires training to fully construct each graph, having substantial resource needs. It is also restricted by limited interpretability, given that tokens are not represented individually, and it requires manually selecting the number of sub-sentences, which can be a convoluted, high-impact hyperparameter (Zhang et al. 2022). Similarly, VISPool is affected by the visibility criterion and the penetrable limit of the VVG, also requiring a high level of domain expertise (Alikasıfoğlu, Aras, and Koc 2024).

To the best of our knowledge, there is no simple, straightforward method that converts the most important information within PLMs to graphs, while functioning without context length limitations.

3. Methodology

Here, we introduce and explain our novel graph construction method that solves the problems in previous work. We first introduce our rationale, followed by a detailed

**Figure 1**

Heat-maps of average-pooled attention coefficients obtained using BERT for a randomly selected document from Ohsumed’s test split.

step-by-step explanation of how to construct a token graph from the outputs of a Transformer. Afterwards, we explain our learning architecture, including detailed notions about the employed GNN layers. We then detail the benchmark data sets and PLMs used in this study.

3.1 Proposed Graph Construction Method

Given how much our method differs from existing work, we start by explaining our rationale before presenting a detailed overview of all steps that constitute our pipeline.

3.1.1 From PLMs to Graphs. Our proposal is based on the idea that the set of attention coefficients in a PLM can be inherently seen as a graph. Figure 1 depicts average-pooled attention coefficients for a randomly selected document from Ohsumed’s¹ test split obtained using BERT. We may look at the x -axis of either sub-figure as the destination node and the y -axis as the starting point of all edges in a hypothetical graph. Using all aggregated attentions (as seen in Figure 1a) would result in an unnecessarily dense, fully connected graph. However, not all joint occurrences of words have the same relevance, e.g., as seen by the different levels of attention. What would happen if we were to only focus on a subset of the attention coefficients to construct the edges of a graph? This is the central aspect of our method.

Figure 1b depicts this scenario by only including the attentions above the 0.95 quantile² after dropping all “to” and “from” relations that contain special tokens (e.g., [SEP], [CLS])—which we found to have some of the highest attention coefficients, but are not relevant to our model-agnostic setting. Compared with the previous fully connected graph (i.e., Figure 1a), the resulting graph from this heat-map would be much sparser, having considerably fewer values and, thus, fewer memory requirements. Furthermore, although most of the highest attentions can be attributed to close distance co-occurrences, there are several exceptions. As a result, we expect our method to capture more relevant edges than co-occurrence graphs since it builds upon co-

¹ <http://disi.unitn.it/moschitti/corpora.htm>.

² User-defined hyperparameter for illustrative purposes.

occurrences. Finally, our technique can be extended to documents with token counts above the maximum sequence lengths of PLMs via a sliding mechanism. This is a direct result of filtering low attention coefficients, which removes the constraint of having fully connected graphs that limits Transformers. However, this may come at the cost of some falsely pruned relevant edges.

3.1.2 Method Overview. Granted that a text is not directly converted to a graph, we must first process it to obtain building blocks (i.e., nodes) that connect through predetermined relationships (i.e., edges). Older approaches, such as MPAD or TextING, do so by, e.g., removing non-alphanumeric characters and contractions, lower-casing, splitting on white-spaces, and removing stop-words. In contrast, our method solely relies on the PLM to pre-process each text. Figure 2 contains an overview diagram of our proposed method. The text is first input to the PLM for inference. Then, token embeddings are used to represent each node and attention coefficients are directly translated into edges. We remove special tokens and all attentions coefficients that relate to them during this stage to make our method model-agnostic. A node may be created for every token (i.e., repeated tokens are treated individually) or we may group the information from repeated tokens (colored in blue) to reduce dimensionality. We call the former option the **surrogate** variant, in which we use the token indices as node identifiers, and the latter is the **grouped** variant, where input identifiers are used to label tokens. The aggregation of the latter is done by pooling over all node embedding dimensions and concatenating edges with pooling only applied to edges that connect to/from the aggregated nodes. Afterwards, edges for which their single-dimension weight (obtained through complete feature aggregation) is below a predetermined threshold are dropped. Lastly, isolated nodes (i.e., nodes that are not connected to any node, including themselves; colored in orange) are removed to avoid node bias during learning.

We designed our method so that it may be used with variable document length, including documents that do not fit inside the PLM’s context window. This is accomplished via a chunk-and-stride mechanism that functions as a sliding window with padding to maintain some context in each chunk. We use the maximum sequence length

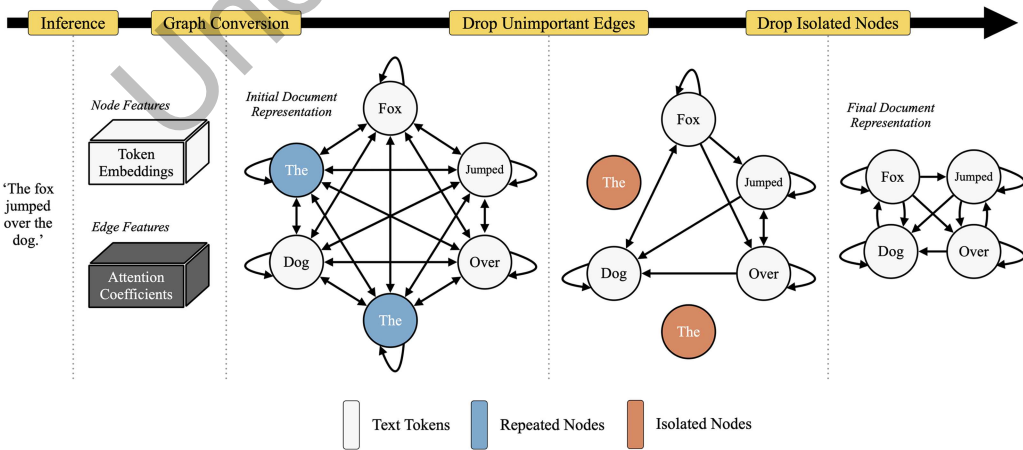
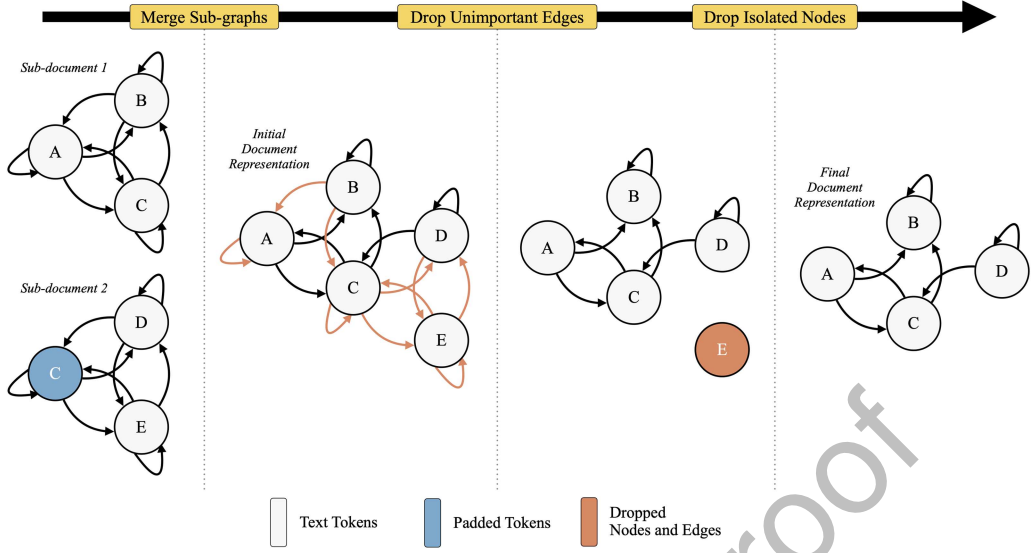


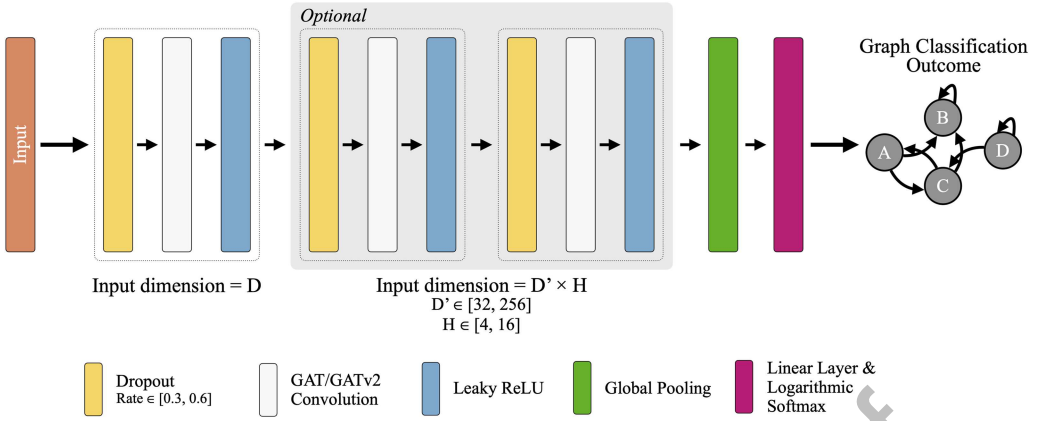
Figure 2
Overview diagram of our proposed method.

**Figure 3**

Overview diagram of the chunk-and-stride mechanism of our proposed method. Sub-graphs (1) (composed by nodes *A*, *B*, and *C*) and (2) (with nodes *C*, *D*, and *E*) consist of direct translations of token embeddings and attention matrices to graphs as node and edge features, respectively. Chunk-level graphs (1) and (2) are combined to produce a document-level graph. Unimportant edges (based on a predetermined threshold) are then dropped, followed by the removal of isolated nodes.

of the PLM as the maximum chunk size, and the left and right strides are values in $\{0, 32, 64, 128, 256\}$. The necessary special tokens are added to the start and end of each chunk so that the PLMs can infer properly. Figure 3 depicts how our method works when using this chunk-and-stride mechanism. Chunks are first created from the input text, followed by inference for each sub-document. The resulting token embeddings and attention coefficients from each chunk are then converted into sub-graphs, followed by a combination operation to obtain a document-level graph. Padded tokens (colored in blue) play a pivotal role during this combination when using the *surrogate* variant, as sub-graphs would not have a way to be connected otherwise. As a result of some duplicate nodes from the overlapped segments, we perform a group-by operation over repeated nodes, similar to the one used in the *grouped* variant. Afterwards, the process is the same as in Figure 2—i.e., unimportant edges and isolated nodes are dropped to obtain the final document representation. In essence, our chunk-and-stride mechanism accomplishes offline, windowed attention sparsification via overlap-based context preservation and post-hoc pruning that induces representational sparsity instead of computational sparsity.

Another unique aspect of our proposal is how edge features are defined. Given that PLMs have a wide range of values to represent the relationships between tokens, we wanted to take full advantage of this information. For each pair of tokens, there are $L \times H$ different attention coefficients that can be used to define F feature values for each edge, where L is the count of model layers and H of attention heads. We evaluated three different levels of edge feature aggregation in this study, each conveying a distinct level of information. We may not perform aggregation ($F = L \times H$), with each value representing the information from an attention head in a specific layer. Another option

**Figure 4**

Model architecture diagram. The displayed model is composed by 3 convolutional blocks, followed by global pooling, and a linear layer.

is to follow a layer-wise aggregation strategy ($F = L$) for which edge features represent layer-level knowledge. Alternatively, we can fully aggregate all attention coefficients for each edge ($F = 1$) to obtain a global relationship importance estimate. For both node and edge features, aggregation consists of minimum, average, or maximum pooling, with this choice being a hyperparameter of our pipeline.

3.2 Model Architecture

In this work, we focus on graph construction, rather than on the model architecture. Our models are composed by up to 5 commonly used GNN layers—specifically only GAT (Veličković et al. 2018) or only GATv2 (Brody, Alon, and Yahav 2022) layers—that are used to progressively update node embeddings. Figure 4 exemplifies our architecture using 3 convolution blocks (i.e., dropout with a rate between 30% and 60% $\rightarrow$ GAT/GATv2 layer $\rightarrow$ leaky ReLU activation function). The input shape of the first layer corresponds to the pre-trained embedding dimension, with the layer’s hidden embedding dimension being a value between 32 and 256 for each attention head ($H \in [4, 16]$). The input and output shapes of all remaining layers have the same size as the first’s output for simplicity purposes. These convolutional blocks are followed by global pooling to obtain a graph-level embedding by performing maximum, average, or sum pooling over all nodes. Lastly, classification is performed by passing the previous embedding through a linear layer and a logarithmic softmax function.

3.3 Benchmark Data Sets

To ensure the consistency and robustness of our novel approach, we select a varied set of heterogeneous data sets: Ohsumed, R8 (a subset of Reuters-21578³), Stanford sentiment treebank (SST)-2⁴ (Socher et al. 2013), and IMDB-1k. SST-2 is based on Rotten Tomatoes

³ <https://archive.ics.uci.edu/dataset/137/reuters+21578+text+categorization+collection>.

⁴ <https://nlp.stanford.edu/sentiment/>.

sentence polarity (RTSP) (Pang and Lee 2005), and was obtained using HuggingFace⁵). IMDb-1k is a subset of the internet movie database (IMDb) (Maas et al. 2011) formed by the top 500 largest documents in each split (train and test), for which length estimates were calculated using the CountVectorizer⁶ method from scikit-learn (Pedregosa et al. 2011). The tasks in the first two data sets are categorization of medical abstracts and news, and the last two comprise sentiment polarity identification of short and long movie reviews, respectively.

Ohsumed, R8, and IMDb-1k lack validation splits so, to avoid any potential overfitting during our experiments, we use a random partition of 20% of the training data of each of these data sets, extracted in a stratified manner to retain the distribution of classes as validation splits. These selections are done using a random seed of 42, which is used as the random seed throughout this work.

Knowing that the distribution of classes heavily affects some aspects of training, in particular the choice of the performance metric to optimize, we choose the metric for each data set independently. Both SST-2 and IMDb-1k are mostly balanced, so we optimize accuracy in all experiments. By contrast, R8 and Ohsumed are heavily imbalanced, so we optimize macro F1-scores with these data sets, instead.

3.4 PLMs

We evaluate four distinct PLMs, namely, bidirectional and auto-regressive transformer (BART) (Lewis et al. 2020),⁷ BERT (Devlin et al. 2019),⁸ robustly optimized BERT pre-training approach (RoBERTa) (Liu et al. 2019),⁹ and XLNet (Yang et al. 2019).¹⁰ Although all rely on attention mechanisms that are crucial to our method, there are considerable differences between the models.

4. Experimental Details and Results

This section starts with identifying our optimal method configuration. We then compare using our method with fine-tuning the PLM instead. This is followed by an evaluation of different PLMs, and a comparison of our best performing configurations with the SotA. Afterwards, we analyze how our method compares to using sliding windows. We then evaluate the generalization capabilities of our proposal. Finally, we assess how the memory needs of our method compare to fine-tuning, as well as sliding widow-based graphs.

4.1 Identifying the Optimal Method Configuration

Figure 5 contains performance ranks from different configurations of our method. As previously stated, the target metric of each data set may differ (e.g., accuracy is used for SST-2, while we optimized macro F1-score for Ohsumed). We use BERT as our baseline PLM and Optuna (Akiba et al. 2019) for hyperparameter tuning. Each performance

⁵ <https://huggingface.co/datasets/sst/viewer/default>.

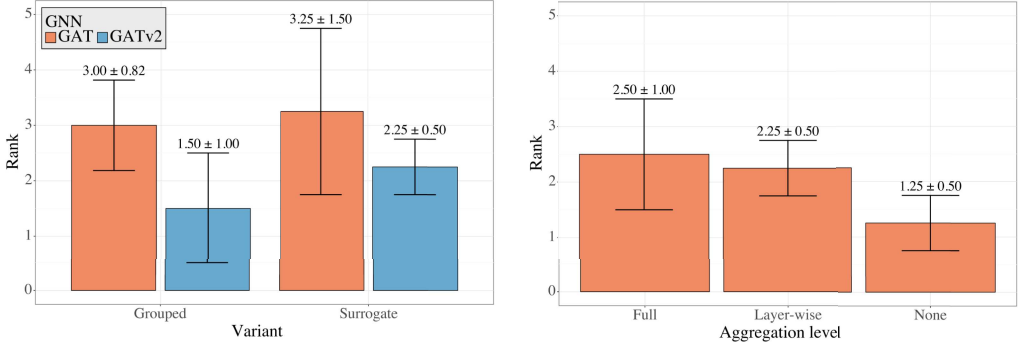
⁶ https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html.

⁷ facebook/bart-base: <https://huggingface.co/facebook/bart-base>.

⁸ google-bert/bert-base-uncased: <https://huggingface.co/google-bert/bert-base-uncased>.

⁹ FacebookAI/roberta-base: <https://huggingface.co/FacebookAI/roberta-base>.

¹⁰ xlnet/xlnet-base-cased: <https://huggingface.co/xlnet/xlnet-base-cased>.



(a) Ranks from different combinations of method variants with GAT- or GATv2-based aggregation obtained by training GATv2-based models trained on BERT token graphs without edge feature aggregation. (b) Ranks from different levels of edge feature aggregation obtained by training GATv2-based models on BERT token graphs following the *grouped* variant.

Figure 5

Average and standard deviation ranks (using the target metrics from each benchmark data set) from training different GNNs on BERT token graphs following the two method variants and different levels of edge feature aggregation. Each bar of Figure 5a is colored based on the GNN.

reported consists of the average and standard deviation from 10 runs with different random seeds ($\in [37, 46]$) of the best trial from a set of 250 successful trials for each configuration.

We begin by exploring if there is an optimal combination of method variant (i.e., *grouped* or *surrogate*) and model architecture (i.e., using GAT or GATv2 layers) in Figure 5a. We train the two model architectures on token graphs constructed using BERT without edge feature aggregation ($F = L \times H$). The results show that using GATv2 layers leads to better results than GAT-based models, regardless of the method variant. Furthermore, learning from graphs built following the *grouped* variant is better on average than the *surrogate* variant. This may not always happen as indicated by comparing the upper-whisker of GATv2 models trained on graphs that follow the *grouped* variant (rank = 2.5) with the mean of similar models trained on the *surrogate* variant (rank = 2.25).

With GATv2-based models trained on graphs built following the *grouped* variant as our first best performing configuration, we explore the impact of different aggregation levels on edge features. Specifically, we compare no-aggregation ($F = L \times H$) with layer-wise-aggregation ($F = L$) and full-aggregation ($F = 1$). As seen in Figure 5b, the results show that no-aggregation consistently leads to the best performance, followed by layer-wise aggregation, and finally full-aggregation. In short, the higher the degree of aggregation (i.e., the more information we lose via merging values), the worse the results.

4.2 Comparing Using Our Method with Fine-tuning the PLM

To understand if and when our approach surpasses fine-tuning, we compare fine-tuned versions of BERT with our two method variants, as seen in Table 1. Fine-tuning leads to the best results for both Ohsumed and SST-2. This may come as a direct result of the extra information used by the pre-trained model in the form of more attention

Table 1

Testing performance metrics (in percentage) for all benchmark data sets from fine-tuning BERT compared with training GATv2-based models on token graphs built using the PLM.

Bold represents the best performance metric value for each data set.

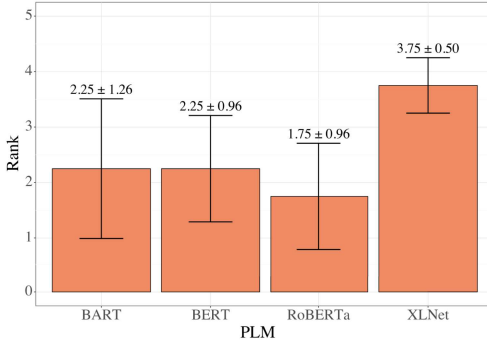
Data Set	Approach	Accuracy	F1-score	Precision	Recall
IMDb-1k	Fine-tuning	82.18 $\pm$ 3.92	82.05 $\pm$ 3.91	82.47 $\pm$ 3.89	82.10 $\pm$ 3.75
	Grouped	92.27 $\pm$ 0.86	92.24 $\pm$ 0.87	92.24 $\pm$ 0.84	92.32 $\pm$ 0.87
	Surrogate	92.36 $\pm$ 0.45	92.34 $\pm$ 0.45	92.33 $\pm$ 0.43	92.46 $\pm$ 0.42
Ohsumed	Fine-tuning	71.90 $\pm$ 0.98	63.00 $\pm$ 1.06	64.63 $\pm$ 0.76	62.79 $\pm$ 1.48
	Grouped	69.96 $\pm$ 0.51	61.35 $\pm$ 0.54	62.20 $\pm$ 0.80	61.65 $\pm$ 0.53
	Surrogate	70.18 $\pm$ 0.83	60.86 $\pm$ 1.16	62.73 $\pm$ 0.92	60.46 $\pm$ 1.17
R8	Fine-tuning	97.78 $\pm$ 0.30	94.37 $\pm$ 0.61	95.09 $\pm$ 0.62	93.96 $\pm$ 0.72
	Grouped	97.85 $\pm$ 0.25	94.66 $\pm$ 0.49	95.02 $\pm$ 0.75	94.47 $\pm$ 0.63
	Surrogate	97.63 $\pm$ 0.33	93.98 $\pm$ 0.53	93.93 $\pm$ 0.84	94.27 $\pm$ 0.58
SST-2	Fine-tuning	91.60 $\pm$ 0.43	91.60 $\pm$ 0.43	91.69 $\pm$ 0.41	91.61 $\pm$ 0.43
	Grouped	89.52 $\pm$ 0.75	89.52 $\pm$ 0.75	89.53 $\pm$ 0.73	89.52 $\pm$ 0.75
	Surrogate	89.18 $\pm$ 0.76	89.17 $\pm$ 0.76	89.21 $\pm$ 0.77	89.18 $\pm$ 0.76

coefficients when compared with our method due to the latter dropping unimportant edges. Furthermore, the vast majority (100% in SST-2 and $\approx 96\%$ in Ohsumed) of documents from these data sets completely fit inside BERT’s context window of 512 tokens. This means that our graphs do not contain extra information in the form of added tokens and attentions. Contrarily, the results on IMDb-1k show clear benefits from using our proposal. Not only is our method considerably superior, having achieved 10.09% and 10.18% higher accuracy for the *grouped* and *surrogate* method variants, respectively, it is also characterized by more consistent performance as seen by smaller standard deviations. We believe this to be explained by how we represent the entire document instead of a small sample of tokens, with the results highlighting that our method is best suited for data sets whose documents are larger than the chosen PLM’s context window. We also compare the distributions of the target performance metrics for each data set from the 10 runs of each approach using unpaired *t*-tests with a confidence level of 95% (i.e., $\alpha = 0.05$). With the exception of the results from R8, the performance differences of fine-tuning BERT and using either variant of our method are statistically significant. R8 is also the only data set for which the performances of the *grouped* and *surrogate* variants are not statistically identical.

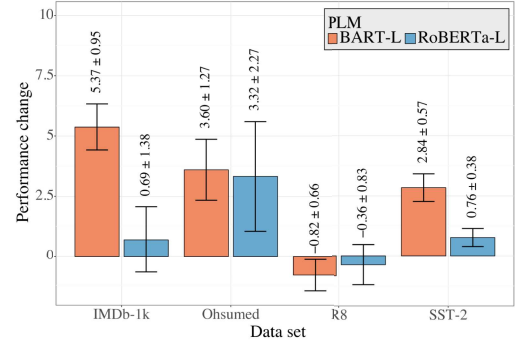
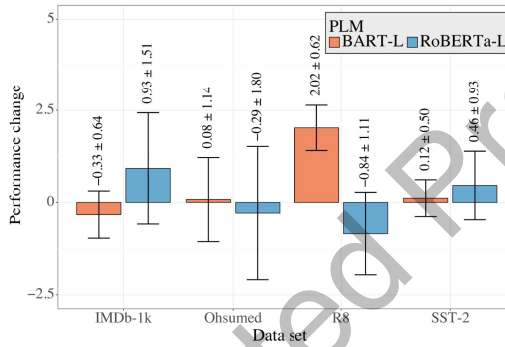
With these results, we show that our graph construction method is best suited for data sets that mostly contain documents that the PLM cannot fully encode, proving that there are benefits in extending the context windows of the PLM via our chunk-and-slide mechanism. For the remaining cases, our method leads to competitive results, but some performance drop may happen.

4.3 PLM Suitability for Graph Construction

We intend to understand how the performance of our method changes when using different PLMs, specifically BART, BERT, RoBERTa, and XLNet. All these pre-trained models are used to construct token graphs that are then used to train models. Both the base construction method and model architecture are the same for all PLMs and for each set of experiments in Figure 6.



(a) Ranks from different PLMs.

(b) Performance change from replacing *base* PLMs with their *large* versions.(c) Performance change from replacing the *grouped* variant with the *surrogate* one.**Figure 6**

Rank and performance change (using the target metric from each benchmark data set) from training GATv2-based models on token graphs constructed using different PLMs and method variants without edge feature aggregation. Each bar in Figure 6b and Figure 6c is colored based on the PLM.

Figure 6a contains average performance ranks from training GATv2-based models using the information from different PLMs following the *grouped* variant without edge feature aggregation. With the exception of XLNet that uses the whole document to calculate token embeddings and attentions, all pre-trained models are defined by a maximum context window that is used for chunking, when necessary. Specifically, 512 tokens for BERT and RoBERTa, and 1024 for BART. RoBERTa is the best performing PLM, followed by BERT and BART with the same average ranks. All three PLMs achieve the best performance in at least one data set, indicating that the choice of a PLM may be problem-dependent. Although BART and BERT have the same average rank, BART's rank is highly influenced by its weaker performance in IMDb-1k compared with the remaining models. We calculate the median ranks of the top 3 PLMs to mitigate this, with the new order being: RoBERTa (Median = 1.5), BART (Median = 2.0), and BERT (Median = 2.5). Given the similarity in the median ranks of both BART and RoBERTa, we choose these two pre-trained models for subsequent analysis.

So that we may assess the impact of higher quality embeddings, as well as denser attention matrices (due to higher layer and attention head counts), we evaluate the

performance difference from replacing *base* with *large* PLMs in Figure 6b. BART-L¹¹ represents the *large* version of BART, and RoBERTa-L¹² of RoBERTa for graph construction. Excluding R8, all changes from *base* to *large* resulted in improved performance. We suspect that these outlier results are a consequence of the lack of punctuation in the documents from this data set. That is, this characteristic may negatively impact both token embeddings and attentions, e.g., leading the model to attribute high importance to sentence-level co-occurrences that may be in no way relevant—each text is composed of a single sentence. The improvements are considerable in some data sets, for example, IMDb-1k and SST-2 using BART-L and Ohsumed using either pre-trained model. Notwithstanding, there are some statistically identical performances, namely, in IMDb-1k and R8 using RoBERTa-L compared to RoBERTa. Overall, we see benefits in using the *large* variants of PLMs alongside our method when the available resources allow it, however, it may not improve performance for all models.

To understand how the previously discarded *surrogate* variant performs alongside other PLMs, we re-evaluate it alongside both BART-L and RoBERTa-L. Figure 6c contains the average performance change from using the *surrogate* variant instead of the *grouped* one for graph construction. Although there are significant performance differences in R8 using both pre-trained models and IMDb-1k using RoBERTa-L, the performances from both method variants in the remaining data sets are statistically identical. These results indicate that the choice between the *grouped* and *surrogate* variants may be data set- and PLM-dependent.

Based on all results, we suspect that the quality of the PLMs is directly linked with that of the resulting graph models, which may be a consequence of our simple model architecture. Our experiments show that BART’s architecture is the most suitable to be used with our method among those evaluated. Intuitively, we expect its summarizing nature to produce graphs that focus on the most important relationships between words. In terms of our method variants, we encourage end-users to first experiment using the *grouped* variant and then the *surrogate* one if enhanced contextual information is of high importance to the task.

4.4 Comparison to the SotA

We compare some of our best performing approaches to the SotA in Table 2 using accuracy results reported in the corresponding publications of each work. Remember that the results we report for our method on Ohsumed and R8 were optimized for macro F1-score instead of accuracy, contrary to most SotA works. When compared with the more traditional inductive approaches (e.g., MPAD and TextING), our method consistently achieves higher accuracy—with a few exceptions in R8, which we attribute to the choice of the optimization metric. We see this as a direct consequence of using richer node embeddings, as well as being able to represent out-of-vocabulary tokens via sub-word segments, instead of randomly initializing these embeddings. Our method also fares considerably well against approaches that use context-aware, pre-trained embeddings. This not only includes inductive techniques, but also ones that learn from a single, corpus-wide heterogeneous graph, such as BERTGCN or RoBERTaGCN. However, as seen before, our proposal is highly dependent on the PLM. Learning from either method variant constructed with BART-L leads to results that regularly surpass existing works,

¹¹ facebook/bart-large: <https://huggingface.co/facebook/bart-large>.

¹² FacebookAI/roberta-large: <https://huggingface.co/FacebookAI/roberta-large>.

Table 2

Accuracy of SotA methods compared to some of our models. **Bold** represents the best performance metric value for each data set, and underlining represents our models that surpass the best SotA method.

	Approach	Ohsumed		R8		SST-2		RTSP	
		Avg.	Max.	Avg.	Max.	Avg.	Max.	Avg.	Max.
State-of-the-art	TextGCN ¹	68.4	–	97.1	–	–	–	76.7	–
	Text-level GNN ²	69.4	–	97.8	–	–	–	–	–
	MPAD ³	–	–	–	97.1	–	87.8	–	–
	TextING ⁴	70.4	–	98.0	–	–	–	79.8	–
	MH-GAT ⁵	–	–	–	–	91.7	–	–	–
	BEGNN ⁶	–	71.2	–	98.4	–	91.4	–	86.4
	BHGAttN ^{7,a}	–	–	–	–	–	–	–	87.7
	BHGAttN ^{7,b}	–	–	–	–	–	–	–	88.9
	HyperGAT ⁸	69.9	–	98.0	–	–	–	78.3	–
	MeHGAT ⁹	72.9	–	97.7	–	–	–	78.1	–
	CoGraphNet ¹⁰	–	73.3	–	–	–	–	–	83.5
	VISPool ^{11,c}	–	–	–	–	–	91.7	–	–
	VISPool ^{11,d}	–	–	–	–	–	93.1	–	–
	BERTGCN ¹²	72.8	–	98.1	–	–	–	86.0	–
	RoBERTaGCN ¹²	72.8	–	98.2	–	–	–	89.7	–
This work	BART-G	71.6	72.5	97.8	98.1	91.9	92.6	86.8	87.1
	BERT-G	70.0	70.9	97.9	98.2	89.5	90.5	–	–
	BERT-S	70.2	71.0	97.6	98.0	89.2	90.6	–	–
	RoBERTa-G	70.9	71.8	97.8	97.9	92.3	92.8	86.5	86.9
	BART-L-G	73.6	74.2	97.4	97.6	94.7	95.4	90.3	90.5
	BART-L-S	73.9	74.3	98.3	98.5	94.8	95.3	89.8	90.2
	RoBERTa-L-G	71.9	<u>73.7</u>	97.7	98.0	<u>93.0</u>	<u>93.5</u>	88.1	88.5
	RoBERTa-L-S	71.2	72.1	97.1	97.5	<u>93.5</u>	<u>94.6</u>	88.2	88.8

¹(Yao, Mao, and Luo 2019) ²(Huang et al. 2019) ³(Nikolentzos, Tixier, and Vazirgiannis 2020)

⁴(Zhang et al. 2020) ⁵(Jin et al. 2022) ⁶(Yang and Cui 2021) ⁷(Zhang et al. 2022) ⁸(Ding et al. 2020)

⁹(Jin et al. 2024) ¹⁰(Li et al. 2025) ¹¹(Alikasıfoğlu, Aras, and Koc 2024) ¹²(Lin et al. 2021).

^aBERT ^bRoBERTa ^cALBERT(Lan et al. 2020) ^dDistil-BERT(Sanh et al. 2019).

yet our learning architecture may be too simple, with the models relying too much on the quality of the PLM. We can see this, for example, by looking at how some works surpass our performance when both our method and the existing work use BERT or RoBERTa as their basis (e.g., BEGNN, BHGAttN, or BERTGCN).

We also compare the official performance of fine-tuning BART-L (Lewis et al. 2020) on SST-2 with that of using our method. As expected from a data set with mostly short documents, fine-tuning does lead to the best results. However, the difference from using our method instead of fine-tuning is considerably small, dropping from 96.6% accuracy to 94.8% (average) or 95.3% (best). This means that all information discarded (i.e., all attention coefficients below the ≈ 0.56 quantile) when constructing the graphs only resulted in a performance decline of 1.8% or 1.3%, respectively. Additionally, our solution works with less than 24GB of memory, which we were unable to make work with fine-tuning BART-L in a timely manner. In addition to documents larger than the

PLM’s context window, we see our method best used in limited-resource scenarios, specifically ones where fine-tuning may be too time consuming.

4.5 Direct Comparison vs. Using Sliding Windows

In order to assess if our method produces more informative graphs when compared with traditional sliding windows, we study how model performance differs between these techniques in Table 3. To make this comparison as fair as possible, we use the same pre-trained embeddings to represent nodes (having extracted them from BART-L), with the set of edges being the key difference between graphs built with our method and with sliding windows. We also use the same model architecture, as well as training and optimization pipelines for both sets of experiments. Each sliding window-based graph is constructed using a sliding window of size $N \in [2, 10]$, centred around each token. Edge weights are then computed as the total count of co-occurrences between every two tokens.

Regardless of data set, at least one of our variants surpasses the performance of models trained using sliding window-based graphs. Overall, sliding windows are defined by the worst average performance rank (2.5 ± 0.58), followed by the *grouped* variant (2.25 ± 0.96), with *surrogate*-based graphs (1.25 ± 0.5) topping the performance in most data sets. The performance gains compared to sliding windows are particularly noticeable in data sets composed by longer texts. Specifically, +1.04% average accuracy for the *grouped* variant and +0.72% for the *surrogate* one in IMDB-1k. +0.97% and +1.04% macro F1-score using *grouped*- and *surrogate*-based graphs in Ohsumed, respectively, and +1.55% macro F1-score in R8 in the scope of the *surrogate* variant. However, not all differences were statistically significant compared to sliding windows, namely the *grouped* variant’s performance in Ohsumed and that of both variants in SST-2.

Table 3

Testing performance metrics (in percentage) for all benchmark data sets from training GATv2-based models on BART-L token graphs constructed following our method variants and traditional sliding windows. **Bold** represents the best performance metric value for each data set.

Data Set	Approach	Accuracy	F1-score	Precision	Recall
IMDb-1k	Grouped	95.49 $\pm$ 0.60	95.47 $\pm$ 0.61	95.45 $\pm$ 0.60	95.52 $\pm$ 0.64
	Surrogate	95.16 $\pm$ 0.13	95.16 $\pm$ 0.13	95.14 $\pm$ 0.13	95.35 $\pm$ 0.13
	Slid. Wind. ¹	94.45 $\pm$ 0.57	94.44 $\pm$ 0.57	94.44 $\pm$ 0.55	94.61 $\pm$ 0.56
Ohsumed	Grouped	73.55 $\pm$ 0.61	66.32 $\pm$ 0.68	66.96 $\pm$ 0.88	67.13 $\pm$ 1.22
	Surrogate	73.85 $\pm$ 0.37	66.40 $\pm$ 0.77	67.16 $\pm$ 0.70	67.04 $\pm$ 0.78
	Slid. Wind. ¹	71.55 $\pm$ 1.12	65.36 $\pm$ 1.30	64.22 $\pm$ 1.86	68.58 $\pm$ 0.76
R8	Grouped	97.39 $\pm$ 0.16	93.56 $\pm$ 0.53	93.71 $\pm$ 0.82	93.60 $\pm$ 0.41
	Surrogate	98.32 $\pm$ 0.08	95.58 $\pm$ 0.33	96.30 $\pm$ 0.43	95.09 $\pm$ 0.37
	Slid. Wind. ¹	97.67 $\pm$ 0.18	94.03 $\pm$ 0.40	94.82 $\pm$ 0.41	93.39 $\pm$ 0.52
SST-2	Grouped	94.70 $\pm$ 0.39	94.69 $\pm$ 0.39	94.75 $\pm$ 0.35	94.70 $\pm$ 0.38
	Surrogate	94.81 $\pm$ 0.37	94.81 $\pm$ 0.37	94.86 $\pm$ 0.36	94.81 $\pm$ 0.37
	Slid. Wind. ¹	94.72 $\pm$ 0.36	94.72 $\pm$ 0.36	94.79 $\pm$ 0.32	94.72 $\pm$ 0.36

¹Sliding windows.

We also notice that, contrary to previous studies that use traditional word embeddings (Nikolentzos, Tixier, and Vazirgiannis 2020; Zhang et al. 2020), most of our best performing sliding window-based models require wide windows. Specifically, the best performing window size in SST-2 and IMDb-1k is 7, and 9 in Ohsumed. The only exception to this is R8, for which the best performing window is of size 2. This pattern indicates that, opposed to traditional word embeddings, transformer-based ones may perform best with wider windows that “simulate”, to some extent, their fully connected inner-workings. This pattern highlights the potential benefits of using our method, since edges use some knowledge that aligns with the PLM’s embedding information—i.e., are constructed using information from the same model as the token embeddings. It also clarifies why shorter documents may not gain as much as longer ones in terms of performance. Even large sliding windows fail to connect distant tokens as the document length increases, and overly large ones result in irrelevant token relationships being considered.

4.6 Generalization Analysis and Ablations

We expand our experiments to additional data sets composed of distinct tasks to assess the generalization capacity of our proposal. Our classification data consists of all previously used benchmarks, except R8 (due to lacking structure in the form of punctuation). There are two regression data sets, namely, Amazon video game reviews (AMZ-VGR)¹³ to predict numeric ratings from user-written reviews, and CommonLit Ease of Readability (CLEAR)^{14,15} (Crossley et al. 2021, 2023) to predict Flesch-Kincaid grade levels from text passages.

We narrow down our possible hyperparameter values to ease comparisons, as well as to have a smaller search space and allow faster convergence. The possible quantile-based thresholds are also stricter, consisting of 0.85, 0.9, 0.95, 0.99, and 0.999. We opt to start at 0.85 despite having some data sets for which the best performing threshold had been lower so that we can evaluate the relationship between sparsity and performance. The remaining graph construction hyperparameters are fixed to speed-up experiments by pre-computing all graphs beforehand. We use 128 tokens as left stride, maximum- and average-pooling for attention and token embedding aggregation, respectively. No edge feature aggregation is performed while constructing the graphs. Additionally, right stride is not used, as we found it to not be as important as left stride. As a result of discretizing many numerical hyperparameters and a risk of converging to local minima, we increase the number of randomly sampled trials from 10 to 150, followed by 100 trials sampled using tree-structured Parzen estimator (TPE) sampler.¹⁶ Duplicate trials are skipped, not accounting for the total trial count.

4.6.1 Model Architecture. In order to understand how downstream performance is influenced when using our method as the basis for graph construction, we compare three model architectures. Our new baseline is a modified version of Figure 4 with a different order of operations within each convolutional block. While previously we had dropout → convolution → leaky ReLU, we now have convolution → leaky ReLU →

¹³ <https://huggingface.co/datasets/kuroneko5943/amz20> (video games).

¹⁴ <https://www.kaggle.com/datasets/verracodeguacas/clear-corpus/data>.

¹⁵ Similarly to IMDb-1k, we created a validation split for the CLEAR data set.

¹⁶ <https://optuna.readthedocs.io/en/stable/reference/samplers/generated/optuna.samplers.TPESampler.html>.

Table 4

Testing performance metrics for all benchmark data sets from training models with distinct architectures on BART-L token graphs constructed following the *surrogate* variant.

Bold represents the best performance metric value for each data set.

Task	Data Set	Metric	Architecture		
			Baseline	Att. Pool.	Hierar. Pool.
Classification	IMDb-1k	Acc. ¹ (%)	95.49 ± 0.45	95.28 ± 0.91	95.28 ± 0.54
	Ohsumed	F1-sc. ² (%)	65.86 ± 0.66	64.46 ± 0.79	64.68 ± 0.84
	SST-2	Acc. ¹ (%)	94.17 ± 0.30	94.51 ± 0.23	94.54 ± 0.17
Regression	AMZ-VGR	RMSE	0.888 ± 0.032	0.983 ± 0.062	0.987 ± 0.143
	CLEAR	RMSE	1.724 ± 0.058	1.919 ± 0.142	2.127 ± 0.191
Paraphrasing Detection	PAWS	Acc. ¹ (%)	73.83 ± 0.61	–	–
Summarization	BBC-NS	ROUGE-2	0.683 ± 0.005	–	–

¹Accuracy. ²F1-score.

dropout. This change not only aims to improve regularization, but also enhance training stability, accelerate convergence, and promote the learning of more robust features. We also add graph normalization layers (Cai et al. 2021) after each dropout operation to smooth gradient flow and stabilize optimization, while also reducing the risk of vanishing and exploding gradients. The second architecture we evaluate uses attention-based global pooling¹⁷ instead of maximum-/average-/sum-pooling of all nodes, i.e., it considers node importance when constructing graph-level embeddings. Lastly, we explore whether progressive, learning-based pruning via self-attention graph pooling (SAGPooling) (Knyazev, Taylor, and Amer 2019; Lee, Lee, and Kang 2019) applied at the end of each convolutional block (i.e., in a hierarchical manner) leads to better performance while further reducing memory needs. Table 4 contains the target performance metrics for each combination of data set and model architecture using BART-L as the basis for graph construction. With the exception of SST-2, the baseline performed the best for all data sets. Unpaired *t*-tests showed that all architectures performed identically in IMDb-1k and that CLEAR is the only data set for which the attention- and the hierarchical-pooling approaches resulted in statistically different performance. We note that the best performances reported in the new experiments are worse than those reported in previous sections. This is due to a more bound hyperparameter search space, limiting the models’ ability to learn all fine-grained information necessary to reach peak performance. A possible solution to this would be to have a 2-step tuning stage, in which we would complement the stricter search space with a subsequent one where parameters are not bound to a set of predetermined values.

The small performance difference between architectures suggests that our graph construction method is also model-agnostic with regard to the learning architecture, i.e., its downstream performance is not heavily impacted by architecture-specific properties. This could be a direct result of using high-quality information from PLMs to represent

¹⁷ https://pytorch-geometric.readthedocs.io/en/1.3.1/_modules/torch_geometric/nn/glob/attention.html.

texts and learn from these representations using networks that follow the same core principals as Transformers, but in a sparser manner. Conversely, these results together with those in Section 5.2 could indicate that our graphs may suffer from (i) unimportant edges kept after pruning and/or (ii) some over-parameterization of edge features that negatively impact performance. Such problems could also justify why the models are more impacted by node-level ablations when compared with edge-based ones (refer to Section 5.2).

We also evaluate our best architecture on two more tasks, namely, paraphrasing detection using paraphrase adversaries from word scrambling (PAWS)¹⁸ (Zhang, Baldridge, and He 2019) and extractive text summarization via BBC news summary (BBC-NS)^{19,20} (Greene and Cunningham 2006). These tasks assess how well positional information is preserved in the graphs given that both require strong word-order modeling. Some modifications to our pipeline are needed. Paraphrasing detection entails receiving two graphs (one for each document), passing each through the GNN encoder, and concatenating both graph-level embeddings before feeding them to a classification layer. In contrast, the summarization task does not have a global pooling stage to construct graph-level embeddings. Instead, a binary mask is constructed to define which tokens from the original text are kept in the summary.

While the results are not as clear as those from classification and regression tasks, our modeling approach is mostly accurate in extracting meaning from complex text on PAWS. We notice some trends in the wrongly classified sentence pairs, namely, some difficulty in dealing with role reversal (e.g., swapped subjects or locations), as well as some confusion with subject-object distinction. Additional experiments on the subject could help mitigate these issues, e.g., by using a more adequate loss function with contrastive properties. The summarization results follow a similar pattern, in which the average performance hints that the model is able to match the reference and candidate summaries closely in content and wording; ROUGE-1 shows a $\approx 75.9\%$ token-level overlap and ROUGE-2 a $\approx 68.3\%$ bi-gram overlap. BERT-score²¹ (a semantic similarity score between candidate and reference summaries) with value $92.41 \pm 0.11\%$ also supports that the models convey the desired semantic information by producing semantically similar summaries compared to the real ones. In contrast, ROUGE-L (measurement of the longest common sequence of words, keeping word order intact), is considerably lower (0.465 ± 0.003), which we attribute to the sentence order changing between some original texts and summaries. As in the previous task, we individually analyze some examples to better understand the model's performance. While many candidate summaries are semantically and syntactically coherent, there are some that fail to grasp the important content in the original texts, despite keeping most of the desired semantic meaning. Although these results show the capabilities of our method to model complex, order-dependent information in text, some changes to the downstream model could be done to improve performance—e.g., adding a sentence reordering module or relying on a PLM to merge the selected tokens in a coherent manner.

18 <https://www.kaggle.com/datasets/thedevastator/the-paws-dataset-for-paraphrase-identification>.

19 <https://www.kaggle.com/datasets/pariza/bbc-news-summary>.

20 Given the lack of an official training/validation/testing split for BBC-NS, we constructed a custom one following a stratified division of topics across splits.

21 <https://huggingface.co/spaces/evaluate-metric/bertscore>.

Table 5
Testing performance metrics for all benchmark data sets from training GATv2-based models on BART-L and LLaMa-3.2-1B token graphs constructed following the *surrogate* variant. **Bold** represents the best performance metric value for each data set.

Task	Data Set	Metric	Language Model	
			BART-L	LLaMa-3.2-1B
Classification	IMDb-1k	Acc. ¹ (%)	95.49 ± 0.45	93.70 ± 0.98
	Ohsumed	F1-sc. ² (%)	65.86 ± 0.66	69.37 ± 0.58
	SST-2	Acc. ¹ (%)	94.17 ± 0.30	91.80 ± 0.35
Regression	AMZ-VGR	RMSE	0.888 ± 0.032	1.091 ± 0.045
	CLEAR	RMSE	1.724 ± 0.058	2.098 ± 0.140
Paraphrasing Detection	PAWS	Acc. ¹ (%)	73.83 ± 0.61	58.04 ± 1.72
Summarization	BBC-NS	ROUGE-2	0.683 ± 0.005	0.445 ± 0.037*

¹Accuracy. ²F1-score.
*Layer-wise aggregation used due to out-of-memory errors when constructing the graphs.

4.6.2 *LLM versus PLM.* To understand how larger, recent language models fare alongside our method, we compare the performance of solely changing the pre-trained model used to construct the graphs in Table 5. We use the best model configuration for each data set obtained with BART-L token graphs, and replace the latter with LLaMa-3.2-1B’s²² (Grattafiori et al. 2024) graphs. As expected, graph construction requires more memory with the increase in token embedding dimension, as well as layer and attention head counts. Consequently, we use an NVIDIA H100 GPU and an Intel Xeon Gold 6442Y CPU to construct the graphs for IMDb-1k, AMZ-VGR, and BBC-NS. In fact, the larger documents in BBC-NS required some operations to be run on the CPU using 128GB of RAM, which shows how much extra memory is required by these large language models. In terms of results, with the exception of Ohsumed, the lighter language model leads to the best performance, with all differences being statistically significant.

Although the performance of the models trained on LLaMa-3.2-1B-based graphs could be improved by using the same hyperparameter tuning setup as that of BART-L, our goal with this experiment is to show that solely using LLMs is not always beneficial. Not only is memory going to be a bigger constraint, the results may also worsen. A possible explanation as to why this happens for most of our benchmark data sets is the higher number of node and edge features, which could result in more difficulty in learning relevant patterns for each task. Interpretability of the graphs is also more complex, which may be prohibitive depending on the problem.

4.6.3 *Assessing the Importance of the Threshold.* So that we may understand how important the attention threshold is and how it impacts performance, we perform an ablation over it, i.e., we isolate its impact on performance. This is done on the best performing configurations from BART-L-based graphs on all data sets by fixing all other hyperparameters. With the exception of BBC-NS, we evaluate all possible discrete

²² <https://huggingface.co/meta-llama/Llama-3.2-1B>.

Table 6

Testing performance metrics for all benchmark data sets from training GATv2-based models on BART-L token graphs constructed following the *surrogate* variant and using different quantile-based thresholds. **Bold** represents the best performance metric value for each data set.

Task	Data Set	Metric	Quantile				
			0.85	0.9	0.95	0.99	0.999
Classification	IMDb-1k	Acc. ¹ (%)	94.93 ± 1.08	95.37 ± 0.91	95.49 ± 0.45*	94.60 ± 0.79	96.00 ± 0.87
	Ohsumed	F1-sc. ² (%)	65.86 ± 0.66*	65.11 ± 0.59	63.34 ± 0.71	60.34 ± 0.59	37.32 ± 1.59
	SST-2	Acc. ¹ (%)	94.17 ± 0.30*	92.92 ± 0.36	89.13 ± 0.39	62.31 ± 0.51	49.56 ± 0.80
Regression	AMZ-VGR	RMSE	0.917 ± 0.055	0.899 ± 0.032	0.888 ± 0.032*	0.966 ± 0.039	1.254 ± 0.045
	CLEAR	RMSE	1.820 ± 0.125	1.713 ± 0.103	1.724 ± 0.058*	1.694 ± 0.096	1.912 ± 0.039
Paraphrasing Detection	PAWS	Acc. ¹ (%)	73.83 ± 0.61*	74.02 ± 1.12	72.28 ± 1.76	56.08 ± 1.13	50.52 ± 5.51
Summarization	BBC-NS	ROUGE-2	0.683 ± 0.004	0.683 ± 0.005*	0.676 ± 0.006	–	–

¹Accuracy. ²F1-score.

*Quantile-based threshold selected by Optuna as the best performing one.

quantile thresholds. We leave out the two highest thresholds for this data set to avoid nodes being dropped during pruning, which is not ideal in an extractive summarization task where every token may matter. Results in Table 6 indicate that using the top 5–15% of edges in each graph tends to perform the best, with stricter pruning leading to a higher chance of performance dropping. However, the results in IMDb-1k and CLEAR show that there are exceptions to this, having achieved their best performance with quantile-based thresholds of 0.999 and 0.99, respectively. An opposite example can be found in SST-2, specifically how its best performing threshold in our initial set of experiments is ≈ 0.56 , which is considerably lower than 0.85. These examples suggest that, although focusing on the top 5–15% of edges may lead to high performance, the ideal threshold for a data set is data-specific. Although we recommend starting with a value within this range, the available resources and data characteristics should also be considered. For instance, using a lower threshold alongside a high number of edge features may result in out-of-memory errors during training and/or under-fitting. In contrast, relying on a threshold higher than ideal increases the chance of over-fitting. Consequently, the threshold should be seen as a high impact hyperparameter and optimized appropriately. These results also hint at the idea that some problems may gain from the extra information propagation in the form of higher edge counts, which may result in more similarity between node embeddings as the learning happens. However, this behaviour is not applicable to all problems, as over-smoothing node embeddings may be problematic in node-level tasks, such as summarization, due to loss of token-specific features.

4.7 Analysis of Memory Needs

Table 7 contains the count of trainable parameters from the best performing models trained for each benchmark data set using different approaches. These consist of fine-tuning, using either variant of our method, or learning from sliding window-based graphs. Compared with fine-tuning, our method consistently produces simpler models, specifically $\approx 40\%$ to $\approx 99.14\%$ smaller than those of fine-tuning for the *surrogate* and $\approx 52.24\%$ to $\approx 96.58\%$ for the *grouped* variants, respectively. When compared to using sliding windows for graph construction, using the *surrogate* variant also produces

Table 7

Count of trainable parameters from the best performing models trained for each benchmark data set using either BERT or BART-L. **Bold** represents the lowest count for each PLM-data set pair.

Data Set	Approach	BERT #Parameters	BART-L #Parameters
IMDb-1k	Fine-tuning	109,483,778	–
	Sliding Windows	–	4,955,894
	Grouped	6,934,202	4,809,026
	Surrogate	1,511,372	403,202
Ohsumed	Fine-tuning	109,499,927	–
	Sliding Windows	–	4,234,298
	Grouped	52,293,649	14,751,392
	Surrogate	5,464,958	2,085,663
R8	Fine-tuning	109,488,392	–
	Sliding Windows	–	70,945,928
	Grouped	3,741,008	11,035,208
	Surrogate	938,920	5,911,508
SST-2	Fine-tuning	109,483,778	–
	Sliding Windows	–	8,646,644
	Grouped	14,404,238	1,677,958
	Surrogate	65,686,658	2,956,502

simpler models, corresponding to $\approx 8.14\%$ to $\approx 49.26\%$ of the size of those from sliding windows. Although using the *grouped* variant has a similar behaviour in both R8 and SST-2, its model in IMDb-1k is similar in complexity to that of sliding windows ($\approx 97.04\%$ of its size), and the one in Ohsumed is actually $\approx 348.38\%$ bigger.

We also analyze the distribution of documents over values required to represent each one. Equation (1) shows how the count of values to represent a document (T) is calculated based on the count of nodes (N) and edges (E), as well as the corresponding embedding dimensions (D_N and D_E , respectively).

$$T = N \times D_N + E \times D_E \quad (1)$$

In Figure 7 we see these distributions for both variants of our method, as well as fine-tuning, all using BERT. Given that fine-tuning does not rely on graphs like our method, we define $E = N \times N$ and $D_E = L \times H$ to account for its fully connected nature, where L is the number of model layers and H of attention heads for the PLM. Regardless of data set, our method is considerably lighter than fine-tuning in terms of memory needs. This is even true for IMDb-1k, where fine-tuning truncates all documents in contrast to our method that fully encodes them. Not only is our method lighter, it also performs better in this particular data set. We show that our lightweight solution is a valid alternative to fine-tuning in limited resource scenarios, having minimal performance drops in shorter documents and the potential of improving performance when used to extract information from longer texts.

We also compare our method with using sliding windows alongside BART-L in Figure 8. We see that sliding windows require fewer values to represent each document than our method for all data sets. This is mainly due to edge embeddings from

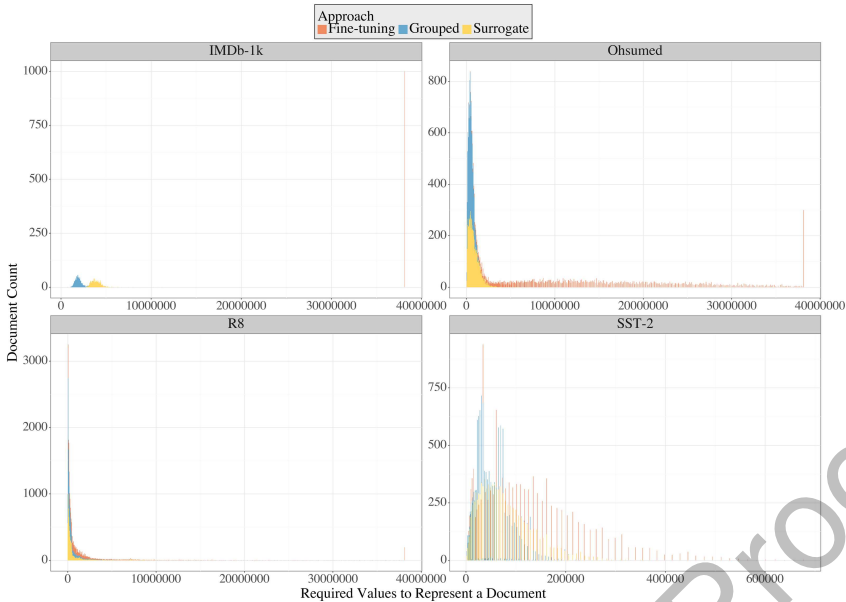


Figure 7
Documents over values required to represent each document comparing both variants of our method with fine-tuning, both using BERT. Vertical lines are colored based on the approach.

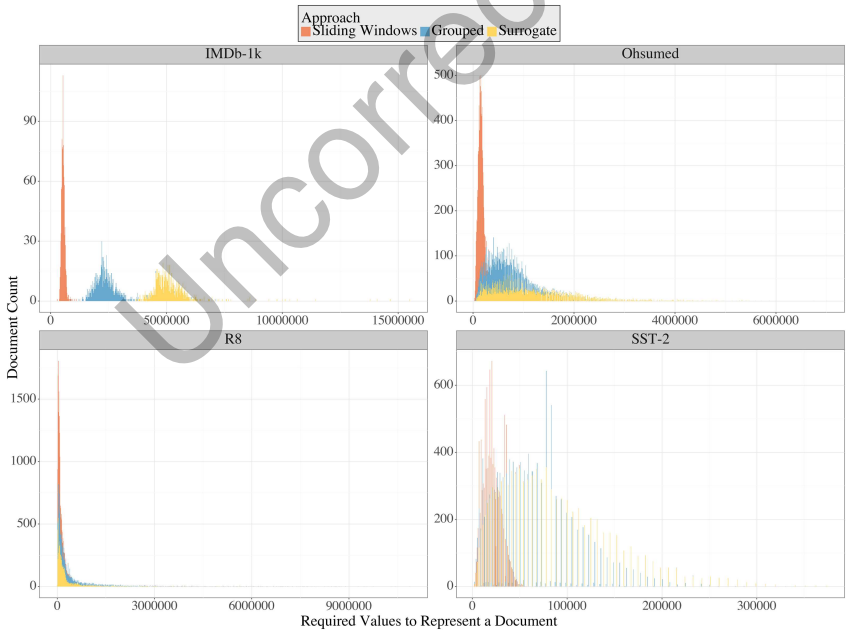


Figure 8
Documents over values required to represent each document comparing both variants of our method with sliding windows for graph construction, both using BART-L. Vertical lines are colored based on the approach.

sliding window-based graphs being uni-dimensional contrarily to the best performing configuration of our method ($D_E = L \times H$). Additionally, as we have seen in Figure 1, our method acts as an extension to sliding windows, since it accounts for distant token relationships in addition to those in narrow co-occurrences (mostly defined by high aggregated attentions). As a result, this behaviour was expected. Nevertheless, even though more values are required when compared to sliding windows, using our method is expected to produce simpler models and better performance, being an improvement over traditional sliding windows for graph construction.

5. Conclusions and Future Work

Fixed-size sliding windows are inherently limited by their size, sometimes ignoring relevant, long-distance relationships between words. We explore how the attention coefficients from PLMs can be used as the basis for a new graph construction method that exploits the semantic and relational information inside these models. By doing so, we mitigate the primary limitation of sliding windows by not restricting the relationships of each token to a narrow interval. Instead, the edges that span from each node are constructed using language-specific knowledge from PLMs. Furthermore, our method can artificially extend the context windows of PLMs so that documents that do not fit inside these windows may be fully encoded, while also reducing overall memory needs.

5.1 Main Takeaways

Our experiments show that using our graph construction method consistently leads to better performance than sliding windows when both approaches are compared under the same model architecture. This behaviour is particularly noticeable in data sets composed of increasingly longer documents, with our method reaching statistically identical performance to that of sliding windows in shorter texts.

Our method also surpasses the performance of GNN-based SotA techniques; however, choosing the most adequate PLM for graph construction is crucial. Additionally, some sliding window-based existing works produce better results than our method when both use the same pre-trained embeddings. While we initially suspected this could be a result of our simple model architecture, our method shows model-agnostic properties in regards to performance. Given that some attention heads in specific layers can be dropped with limited performance impact for the PLMs (Voita et al. 2019; Michel, Levy, and Neubig 2019), our graphs may have over-parameterized edge features. This is a direct result of having many values impacting message passing that are not relevant to information propagation. These values lead to noise in the graphs that is progressively and consistently propagated, damaging the learning cycle. This could also explain why our models are more impacted by node-level ablations than by edge-based ones, with node embeddings being richer than edge ones.

When compared to fine-tuning, models trained on graphs constructed using our method reach comparable performance in documents that fit inside the PLM's context window. In contrast, our method considerably surpasses fine-tuning (with improvements between 10.09% and 10.18% average accuracy) in documents that do *not* fit inside this window, which comes as a result of fully encoding each document. Despite improvements in increasing context size in language models, the strength of our proposal is its sparsity. Our method is also characterized by fewer trainable parameters than fine-tuning, which may be key when used alongside small data sets. Therefore, our method

is more easily used in low-resource scenarios, as not every end-user has the means to fine-tune a PLM.

Lastly, both our method variants show similar performance, with the best choice for each problem being PLM- and data-specific. While the *grouped* variant is characterized by fewer values in each graph, its models tend to be more complex. By contrast, the *surrogate* variant has more values per graph, but produces models with fewer trainable parameters. We find BART-L to be the best match for our method, which we suspect is a result of its higher quality token embeddings, and wider context window.

5.2 Future Directions

We are interested in exploring the best ways to take advantage of the information inside prompt-based LLMs, such as LLaMa-3.2, for graph construction. For example, would the best results be obtained by simply feeding the documents to the model without any task-specific information, or by performing zero- or few-shot inference via prompting? We expect the experiments with this type of models to provide additional understanding about the relational knowledge within PLMs, including uncovering which changes occur in the attention coefficients from prompts alone.

We also plan to explore ways to make graphs sparser while improving their interpretability. Attention heads in different layers attend to different things (Clark et al. 2019). For example, some attend to word relationships such as noun modifiers, while others attend to potentially less relevant aspects like periods or the next token. However, even these arguably less important relationships can bias the final set of edges if their corresponding aggregated value is above that of the threshold. Additionally, we will explore how token-specific pruning strategies that rely on attention information (e.g., Kim et al. 2022) can be used as a part of our pruning step. We expect these changes to help deal with edge feature over-parameterization, making node and edge feature importance more balanced during model training.

Table A.1
List of packages and respective versions used during the course of this work.

Package	Version	Version (Section 4.6)
PyTorch	2.2.2+cu118	2.7.1+cu128
PyTorch Geometric	2.5.3	2.6.1
HuggingFace	4.40.1	4.53.0
TensorFlow	2.14	–
Scikit-learn	1.4.2	1.6.1
Numpy	1.26.3	2.2.2
Pandas	2.2.2	2.2.3
SciPy	1.12	–
Optuna	3.6.1	4.1.0
Plotnine	0.13.5	0.14.1

Appendix A: Hardware and Software Details

Most experiments are performed on a SLURM²³ cluster using GPU-enhanced computing nodes with NVIDIA a40 GPUs and Intel Xeon Silver 4110 CPUs. A single CPU and GPU are used for most experiments with a maximum of 24GB of RAM. Due to maintenance on our computational cluster, the experiments in Section 4.6 each use a single NVIDIA L40S GPU and Intel Xeon Gold 6338 CPUs, with a few exceptions properly documented. We also parallelize data loading by using a total of 9 CPUs in these experiments, with a total of 72GB of RAM shared between all.

In terms of software, we use Python 3.10.4 alongside the packages in Table A.1. Experiments in Section 4.6 use Python 3.13.2, instead.

All code developed for this work, analyses, and examples mentioned in the text can be found at https://github.com/citoplasme/PLM_token_graphs and https://github.com/citoplasme/PLM_token_graphs_extra_experiments.

Appendix B: Data Set Properties

Table B.1 contains some statistics about each data set. With the exception of text length and type-token ratio (TTR) (a measure of lexical richness (LR) that indicates the percentage of unique words in each document) that are obtained using scikit-learn, the remaining metrics are calculated using textstat.²⁴ We see a pattern of decreasing average document length and word repetition from top-to-bottom in the classification data sets, with SST-2 being defined by very short texts with almost no repeated words, in contrast to IMDB-1k which is composed by long movie essays with several duplicate words. The average sentence length of all data sets is relatively similar, with the exception of R8. This is due to this data set having no sentence delimiters, solely containing

²³ <https://slurm.schedmd.com>.
²⁴ <https://pypi.org/project/textstat/>.

Table B.1
Text properties of the benchmark data sets.

Task	Data Set	Split	Text Length			Sentence Length (Avg. $\pm$ Std.)	TTR (Avg. $\pm$ Std.)	Reading Ease Score (Median)
			Min.	Avg. $\pm$ Std.	Max.			
Classification	IMDb-1k	Train. ¹	770	907.7 $\pm$ 143.0	2386	22.5 $\pm$ 6.4	0.5 $\pm$ 0.0	Standard
		Val. ²	755	870.4 $\pm$ 78.1	1047	22.4 $\pm$ 6.8	0.5 $\pm$ 0.0	Standard
		Test. ³	754	894.4 $\pm$ 152.5	2271	22.3 $\pm$ 6.1	0.5 $\pm$ 0.0	Standard
	Ohsumed	Train. ¹	31	181.8 $\pm$ 69.3	420	19.0 $\pm$ 4.9	0.6 $\pm$ 0.1	Difficult
		Val. ²	34	182.6 $\pm$ 68.7	418	19.1 $\pm$ 5.0	0.6 $\pm$ 0.1	Difficult
		Test. ³	27	183.1 $\pm$ 73.7	597	19.1 $\pm$ 5.0	0.6 $\pm$ 0.1	Difficult
	R8	Train. ¹	5	101.7 $\pm$ 116.1	887	101.7 $\pm$ 116.1	0.7 $\pm$ 0.2	Very Confusing
		Val. ²	5	99.2 $\pm$ 112.2	772	99.2 $\pm$ 112.2	0.7 $\pm$ 0.2	Very Confusing
		Test. ³	7	91.2 $\pm$ 102.7	806	91.2 $\pm$ 102.7	0.7 $\pm$ 0.1	Very Confusing
	SST-2	Train. ¹	1	16.3 $\pm$ 8.3	47	15.8 $\pm$ 8.2	0.95 $\pm$ 0.1	Standard
		Val. ²	1	16.6 $\pm$ 7.8	43	16.1 $\pm$ 7.8	0.95 $\pm$ 0.1	Standard
		Test. ³	1	16.3 $\pm$ 8.0	50	15.8 $\pm$ 7.9	0.95 $\pm$ 0.1	Standard
Regression	AMZ-VGR	Train. ¹	8	152.2 $\pm$ 234.3	2858	15.6 $\pm$ 7.3	0.7 $\pm$ 0.1	Fairly Easy
		Val. ²	17	155.1 $\pm$ 166.8	807	14.8 $\pm$ 6.0	0.7 $\pm$ 0.1	Easy
		Test. ³	2	168.3 $\pm$ 280.9	2965	14.8 $\pm$ 6.6	0.7 $\pm$ 0.2	Fairly Easy
	CLEAR	Train. ¹	127	168.9 $\pm$ 16.8	204	21.0 $\pm$ 8.8	0.6 $\pm$ 0.1	Standard
		Val. ²	125	168.7 $\pm$ 17.6	200	20.8 $\pm$ 9.2	0.6 $\pm$ 0.1	Standard
		Test. ³	128	169.5 $\pm$ 17.00	204	20.5 $\pm$ 8.6	0.6 $\pm$ 0.1	Standard
Paraphrasing Detection	PAWS*	Train. ¹	3	17.9 $\pm$ 4.8	33	17.2 $\pm$ 5.2	0.9 $\pm$ 0.1	Fairly Difficult
		Val. ²	4	18.0 $\pm$ 4.9	33	17.2 $\pm$ 5.3	0.9 $\pm$ 0.1	Fairly Difficult
		Test. ³	4	18.0 $\pm$ 4.8	32	17.2 $\pm$ 5.2	0.9 $\pm$ 0.1	Fairly Difficult
Summarization	BBC-NS	Train. ¹	87	380.8 $\pm$ 237.3	3331	19.5 $\pm$ 3.1	0.6 $\pm$ 0.1	Fairly Difficult
		Val. ²	114	373.9 $\pm$ 246.1	4301	19.7 $\pm$ 2.8	0.6 $\pm$ 0.1	Fairly Difficult
		Test. ³	117	363.1 $\pm$ 192.9	2362	19.6 $\pm$ 3.0	0.6 $\pm$ 0.1	Fairly Difficult

¹Training. ²Validation. ³Testing.
*Values only shown for the original sentences due to candidate’s values being nearly identical.

word/acronym sequences, such as “whirlpool corp st qtr shr cts vs cts” or “monolithic memories inc mmic o rd qtr net qtr ended june...”. This heavily affects the Flesch reading ease score of the documents, which resulted in a median score below 30 (“Very Confusing”). Although with a more natural structure, Ohsumed’s documents are also characterized by a high level of complexity, having a reading ease score in [30, 50] (“Difficult”). In contrast, both median reading scores from the sentiment polarity corpora are “Standard” ([60, 70]), which may be explained by most of the texts being written without the need for domain-specific knowledge. AMZ-VGR is composed by a wide range of documents lengths, with most texts being “Fairly Easy” ([70, 80]). All documents in CLEAR have roughly the same length and a “Standard” reading ease. Differently, both PAWS and BBC-NS are composed by “Fairly Difficult” ([50, 60]) documents, which can be related to the complexity of their tasks. In terms of length, PAWS is similar to SST-2 which can be attributed to its entries being sentence pairs. In contrast, BBC-NS is more similar to AMZ-VGR, with document sizes spanning a wide range of values.

Appendix C: Hyperparameters

Table C.1 contains the ranges of values used during the Optuna-based optimizations in this study. Since some of these ranges differ for each approach (e.g., our method compared to fine-tuning), we present the values used for each one. We use the AdamW²⁵ optimizer and cross-entropy loss²⁶ for all experiments. Class weights may be used to influence the loss function, for which we use scikit-learn’s method²⁷ with the whole training data to compute the weights. *Plateau patience* refers to the number of epochs without improvements over the target metric until reducing the learning rate by a factor of *plateau divider*. Similarly, early stopping consists of waiting N epochs after reaching the best validation performance (for which the corresponding model state is loaded for the final evaluation via a checkpoint) before stopping the training preemptively. Finally, *global pooling*, used in both graph-based pipelines, comprises the aggregation operation to generate a graph-level embedding from all node embeddings. It consists of computing the sum,²⁸ average,²⁹ or maximum³⁰ of all embedding dimensions.

Data set-specific hyperparameters can be found in Table C.2. Note that RSTP only contains values for our method since we did not perform any experiments using the remaining approaches. This data set is not part of our set of benchmark data sets. Instead, it was added as an extra comparison to the SotA. Also, only our method uses the attention threshold parameter, reason why the table lacks values for the remaining techniques.

The discrete hyperparameter ranges used in the experiments enumerated in Section 4.6 can be seen in Table C.3. In contrast to the remaining tasks, summarization uses binary cross-entropy loss³¹ on the node level. Additionally, we replace the plateau divider learning rate scheduler with cosine annealing with warm restarts (Loshchilov and Hutter 2017) to help escape local minima and converge more quickly. A warm-up phase is also added to the early stopping mechanism to avoid stopping too early.

As a result of ablating over different quantile-based attention thresholds, which requires a wide range of memory usage, we opted to rely on gradient accumulation so that model parameter updates were performed with a consistent effective batch size comparable to the best performing one from previous experiments. These values can be found in Table C.4.

²⁵ <https://pytorch.org/docs/stable/generated/torch.optim.AdamW.html>.

²⁶ <https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html>.

²⁷ https://scikit-learn.org/stable/modules/generated/sklearn.utils.class_weight.compute_class_weight.html.

²⁸ https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.nn.pool.global_add_pool.html.

²⁹ https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.nn.pool.global_mean_pool.html.

³⁰ https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.nn.pool.global_max_pool.html.

³¹ <https://docs.pytorch.org/docs/stable/generated/torch.nn.BCELoss.html>.

Table C.1

Hyperparameters and respective ranges of values used during the Optuna-based optimization.

Hyperparameter	Possible Values		
	Our Method	Sliding Windows	Fine-tuning
Left Stride ¹	{0, 32, 64, 128, 256}	{0, 32, 64, 128, 256}	–
Right Stride ¹	{0, 32, 64, 128, 256}	{0, 32, 64, 128, 256}	–
Window Size ¹	–	[2, 10]	–
Attention Pooling ¹	{min, mean, max}	–	–
Co-occurrence Pooling ¹	–	sum	–
Embedding Pooling ¹	{min, mean, max}	{min, mean, max}	–
Hidden Dimension ²	[32, 256]	[32, 256]	–
Dropout Rate ²	[0.3, 0.6]	[0.3, 0.6]	–
Attention Heads ²	[4, 16]	[4, 16]	–
Hidden Layers ²	[0, 4]	[0, 4]	–
Epochs ²	[50, 200]	[50, 200]	[3, 15]
Global Pooling ²	{mean, sum, max}	{mean, sum, max}	–
Early Stopping Patience ³	[10, 25]	[10, 25]	[2, 5]
Plateau Patience ³	[10, 25]	[10, 25]	[2, 5]
Plateau Divider ³	[2, 10]	[2, 10]	[2, 10]
Learning Rate ⁴	[1e-5, 1e-1]*	[1e-6, 1e-3]*	[1e-5, 1e-4]*
Weight Decay ⁴	[1e-6, 1e-3]*	[1e-6, 1e-3]*	[1e-6, 1e-3]*
β_0 ⁴	[0.8, 0.9]*	[0.8, 0.9]*	[0.8, 0.9]*
β_1 ⁴	[0.98, 0.999]*	[0.98, 0.999]*	[0.98, 0.999]*
ϵ ⁴	[1e-8, 1e-4]*	[1e-8, 1e-4]*	[1e-8, 1e-4]*
Balanced? ⁵	{True, False}	{True, False}	{True, False}

¹Graph Construction. ²Network Parameters. ³Over-fitting Prevention. ⁴Optimizer Parameters.⁵Loss Function.

*Logarithmic Range.

Table C.2

Data set-dependent hyperparameters and respective ranges of values used during the Optuna-based optimization.

Data Set	Hyperparameter	Possible Values		
		Our Method	Sliding Windows	Fine-tuning
IMDb-1k	Attention Threshold	[0.9, 0.99999]	–	–
	Batch Size	[16, 64]	[16, 64]	[8, 32]
Ohsumed	Attention Threshold	[0.8, 0.99999]	–	–
	Batch Size	[32, 128]	[32, 128]	[8, 32]
R8	Attention Threshold	[0.65, 0.99999]	–	–
	Batch Size	[32, 256]	[32, 256]	[8, 32]
SST-2	Attention Threshold	[0.4, 0.95]	–	–
	Batch Size	[32, 256]	[32, 256]	[8, 32]
RTSP	Attention Threshold	[0.4, 0.95]	–	–
	Batch Size	[32, 256]	–	–

Table C.3

Hyperparameters and respective ranges of values used during the Optuna-based optimization of experiments in Section 4.6.

Hyperparameter	Possible Values
Left Stride ¹	128
Right Stride ¹	0
Attention Pooling ¹	max
Embedding Pooling ¹	mean
Attention Threshold ¹	{0.85, 0.9, 0.95, 0.99, 0.999}
Hidden Dimension ²	{16, 32, 64, 128, 256}
Dropout Rate ²	{0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6}
Attention Heads ²	[1, 2, 4, 6]
Hidden Layers ²	[0, 2]
Epochs ²	200
Global Pooling ²	mean
Early Stopping Patience ³	10
Early Stopping Start Epoch ³	10
Linear Warmup Epochs ³	5
Linear Warmup Start Factor ³	0.1
Cosine Annealing Minimum Learning Rate ³	1e-6
Cosine Annealing T0 ³	10
Cosine Annealing Factor ³	1
Learning Rate ⁴	{1e-7, 5e-7, 1e-6, 5e-6, 1e-5, 5e-5, 1e-4, 5e-4, 1e-3, 5e-3, 1e-2}
Weight Decay ⁴	{1e-5, 5e-5, 1e-4, 5e-4, 1e-3, 5e-3, 1e-2, 5e-1, 1e-1}
β_0 ⁴	0.9
β_1 ⁴	0.99
ϵ ⁴	1e-8
Balanced? ⁵	{True, False}

¹Graph Construction. ²Network Parameters. ³Over-fitting Prevention. ⁴Optimizer Parameters.

⁵Loss Function.

Table C.4

Data set-dependent hyperparameters and respective ranges of values used during the Optuna-based optimization of experiments in Section 4.6.

Data Set	Batch Size	Gradient Accumulation Steps
IMDb-1k	4	8
Ohsumed	64	4
SST-2	256	—
AMZ-VGR	8	4
CLEAR	128	2
PAWS	256	—
BBC-NS	16	8

Appendix D: GNN Layers

Our model architecture comprises commonly used GNN layers, specifically GAT (Veličković et al. 2018) and GATv2 (Brody, Alon, and Yahav 2022).

Appendix D.1 GAT

GATs (Veličković et al. 2018) consist of attention-based layers that are useful to deal with variable input sizes, focusing on the most relevant values of a sequence. Given that different importance values may be given to different nodes in a neighborhood, the attention scores may be used to highlight the most relevant values for a given query, improving model interpretability. These also permit full parallelization of computation, and do not require prior information about the global structure of the graph or the features of its nodes (Liu, Feng, and Hu 2022; Veličković et al. 2018). A shared linear transformation defined by a weight matrix (W) is applied to every node as the first step of this layer. Subsequently, attention coefficients (e) are calculated by performing self-attention on the hidden states of the nodes (h) using a shared attention mechanism (α). Equation (D.1) summarizes this process, where e_{ij} constitutes the importance of the feature values of node j towards node i , being only calculated if j is part of the neighborhood $\mathcal{N}(i)$ of node i , i.e., following a masked attention mechanism (Veličković et al. 2018).

$$e_{ij} = \alpha(W h_i, W h_j) \quad (\text{D.1})$$

To allow for direct comparison, attention coefficients are then normalized across all possible neighbors of each node using softmax, as seen in Equation (D.2) (Veličković et al. 2018).

$$a_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})} \quad (\text{D.2})$$

Once all normalized attention scores (a_{ij}) have been calculated, the hidden states of the nodes (h_i) are calculated as a linear combination of all embeddings in each neighborhood (Equation (D.3)).

$$h'_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} a_{ij} W h_j \right) \quad (\text{D.3})$$

The stability of the learning process is increased by relying on multi-head attention (Vaswani et al. 2017), i.e., by having multiple independent attention mechanisms that execute the previous operations and then merge their results at the end of each layer. For all layers but the final one, the hidden states produced by the attention heads are concatenated. In the final layer, these are averaged out and the final non-linearity function (σ) is only applied after this step (Veličković et al. 2018).

Appendix D.2 GATv2

GATv2 (Brody, Alon, and Yahav 2022) is a variant of GAT that modifies the order of operations in the calculation of attention scores so that distinct rankings of those scores can be computed for different nodes. This means that attentions are affected by the query node, while maintaining the time-complexity of the architecture. In GAT, because the learned layers (W) and the attention scoring function (α) are consecutive, these can be collapsed into a single layer, as seen in Equation (D.4).

$$e_{ij} = \sigma(\alpha^\top \cdot [Wh_i || Wh_j]) \quad (\text{D.4})$$

By contrast, GATv2 (Equation (D.5)) applies the attention scoring function after the non-linear function (σ), and the weighing matrix after concatenation, resulting in a neural network that computes the attention score for each query node (Brody, Alon, and Yahav 2022).

$$e_{ij} = \alpha^\top \sigma(W \cdot [h_i || h_j]) \quad (\text{D.5})$$

Appendix E: Step-by-step Method Overview

Algorithm 1 complements Figure 2 and Figure 3, acting as a detailed overview of all steps needed to convert a text into a graph of tokens using our method.

Appendix F: Example Graphs

Figure F.1 and Figure F.2 help visualize some graphs constructed using our method. These contain example graphs for the sentence “*The quick brown fox jumps over the lazy dog*” built using our *surrogate* and *grouped* variants, respectively.

Starting with the *surrogate* variant, Figure F.1a consists of a fully connected graph built without the removal of unimportant edges. When contrasted to Figure F.1b, which only depicts edges with weights above the 0.60 quantile, we clearly see the importance of the thresholding stage to increase sparsity and avoid over-smoothing problems during learning from overly dense graphs. We also see that some edges are more relevant than others. For example, nodes “jumps” and “over” (compound action), “brown” and “fox”, “lazy” and “dog” (entity descriptions), “fox” and “jumps”, and “over” and “dog” (relationship between entities), all have a high importance relationships—as we would intuitively expect. However, we can still find edges with high weights that relate to nodes that are not as relevant to humans (e.g., edges that connect to either “the” nodes). Interestingly, having two distinct “the” nodes (one for each time the token appears in the sentence) is what makes the *surrogate* variant of our method special, since each token is treated individually, unlike the *grouped* technique or most previous related works (e.g., MPAD or TextING).

Figure F.2 contains the same example, but following the more traditional, *grouped* methodology. Again, not resorting to thresholds (Figure F.2a) results in excessively dense graphs, in contrast to filtering out some unnecessary information, as seen in Figure F.2b. When compared with the *surrogate* versions of the graphs, the *grouped* ones appear even sparser, which is a direct consequent of having fewer nodes. Although the graphs appear almost identical due to following similar construction steps, the set of edges differs (e.g., “jumps” now also relates to “dog”). This indicates that graphs built

Algorithm 1 Step-by-step overview of the proposed graph construction method.

1. Tokenize the text and obtain the special token masking;
 2. Set the PLM to evaluation model;
 3. Compute the chunk size: PLM's maximum sequence length – left stride – right stride – 2 **OR** document length **IF** document length $\leq$ PLM's maximum sequence length;
 4. Initialize an empty list to store all chunk-level token embeddings and another for attentions;
 5. Compute attention coefficients and token embeddings for each chunk:
 - (a) Chunk the input sequences and add special tokens to the start and end of each chunk;
 - (b) Without computing gradients, obtain attentions and token embeddings from the PLM;
 - (c) Set node indices: document-level indices **IF** surrogate **ELSE** chunk-level token identifiers;
 - (d) Expand the node indices into a source and target matrix;
 - (e) Append the attentions and token embeddings alongside their corresponding token identifiers to the previously initialized lists;
 6. Concatenate all edge index, edge attribute, node index, and node attribute chunk-level lists into graph-level lists;
 7. Compute the edge embeddings by aggregating the attention coefficients across all chunks via pooling;
 8. **[Optional]** Reduce the edge embedding dimension based on aggregation level (None, Layer-wise, or Full aggregation);
 9. Apply the threshold to drop unimportant edges:
 - (a) Pool all edge features into a single value for each edge via pooling;
 - (b) Compute the N quantile of the pooled edge features;
 - (c) Drop all edges whose pooled attention is below the quantile;
 10. Aggregate the token embeddings across all chunks via pooling;
 11. Update the node identifiers using PyTorch Geometric (PyG) compatible node identifiers;
 12. Drop isolated nodes;
 13. Instantiate a PyG graph object using the final node and edge lists.
-

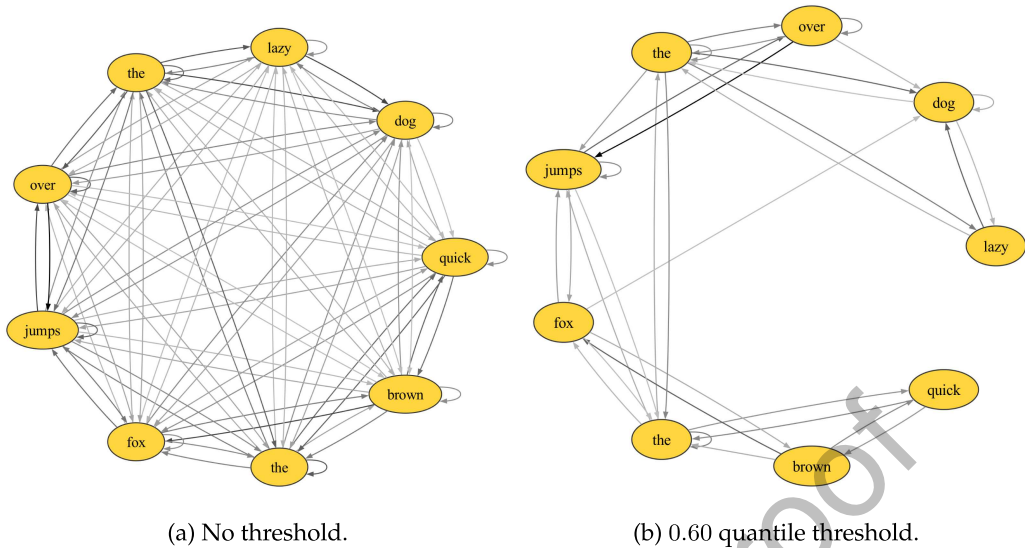


Figure F.1
Example of BERT-based graph representations for the text “The quick brown fox jumps over the lazy dog” constructed using the *surrogate* variant of our method. Edges are colored based on their weights (i.e., fully aggregated attention coefficients), where darker lines indicate stronger relationships. This figure was generated using Graphviz (Ellson et al. 2002).

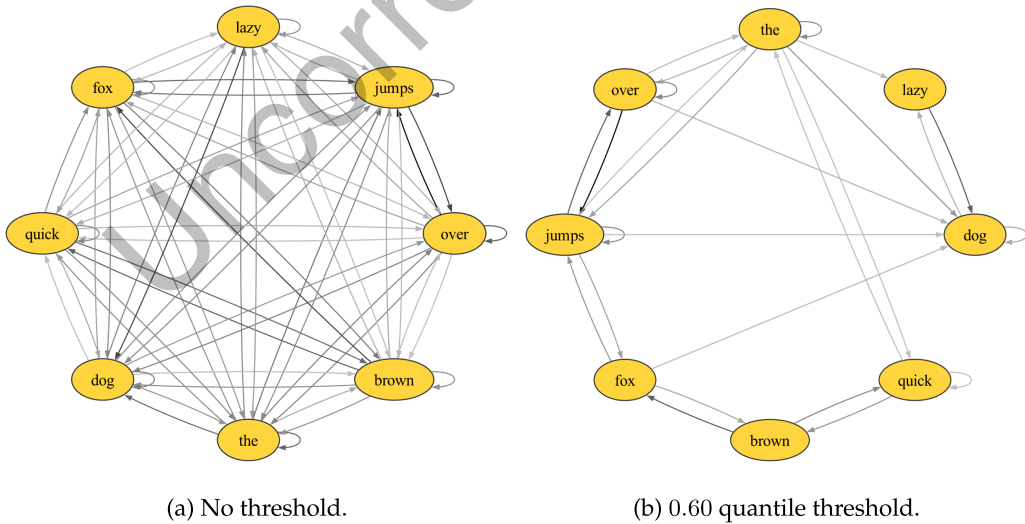


Figure F.2
Example of BERT-based graph representations for the text “The quick brown fox jumps over the lazy dog” constructed using the *grouped* variant of our method. Edges are colored based on their weights (i.e., fully aggregated attention coefficients), where darker lines indicate stronger relationships. This figure was generated using Graphviz (Ellson et al. 2002).

with the two method variants focus on distinct aspects of text. The *surrogate* variant mimics Transformers in giving different meanings to different occurrences of the same words, completely accounting for the textual context of all tokens. Contrarily, graphs built using the *grouped* variant focus on how all occurrences of one word relate to others, i.e., how “keywords” relate.

These visual representations can also be used as a standalone explainability tool to analyze the relationships between tokens similar to BERTViz (Vig 2019). However, it is essential to use an interactive interface to select different edge features (i.e., to subset model layers and attention heads) *ad hoc*. We plan to develop such interface in the future. This comes as a result of the potential for unwanted smoothing or bias during aggregation in the threshold stage that may result in relevant edges being falsely pruned or irrelevant ones being given more importance than deserved. Although this may also influence the performance of downstream models, a modification to our model architecture so that edge features are attributed different weights during learning may solve the problem. Another possible solution would be to subset model layers and/or attention heads to filter the most important information for the problem during graph construction, however, domain- and model-specific knowledge is required.

Appendix G: Additional Results

This section contains details about additional experiments, such as studying how performance relates to text length, ablation studies over node and edge features, and analyzing the training times of different approaches.

Appendix G.1 Performance Relationship with Text Length

We start by studying how performance relates to text length using error dependence plots (EDPs) (Areosa and Torgo 2019, 2020). We discretize the previously obtained text length estimates using quantiles ($N = 4$), as suggested. However, we also want to understand how performance changes for texts that do not fit inside the PLM’s context window. Therefore, we manually add extra cutting points for data sets that have documents that exceed these windows, specifically IMDB-1k, Ohsumed, and R8. Depending on the PLM, the window size differs: 512 tokens for BERT and 1024 for BART-L. We calculate how the text length estimates relate to the tokens generated by each PLM. Regardless of pre-trained model, the text length estimates correspond to $\approx 71\%$ of the token count of each document. We use this value to calculate approximate cutting points for each PLM, namely 361 for BERT and 725 BART-L, already accounting for the start and end special tokens that are not relevant for our analysis.

Figure G.1 contains EDPs for fine-tuning BERT (Figure G.1a) on IMDB-1k, as well as for the *grouped* (Figure G.1b) and *surrogate* (Figure G.1c) variants of our method. No document in this data set completely fits BERT’s maximum sequence length, so we do not add extra cutting points to the discretisation. Regardless of variant, our method produces more accurate results for all text lengths. We see a pattern in all approaches for which the shortest documents have the highest error rate, followed by the longest ones. This could mean that both of these sets of documents are more complex than the remaining ones, regardless of their size.

We compare using sliding windows (Figure G.2a) for graph construction with using our method (Figure G.2b for the *grouped* and Figure G.2c for the *surrogate* variants, respectively) in Figure G.2. This comparison is performed using BART-L on IMDB-1k and, again, no document is expected to fit inside the PLM’s context window ($753 > 725$). When compared with the *grouped* variant, using sliding windows is always less

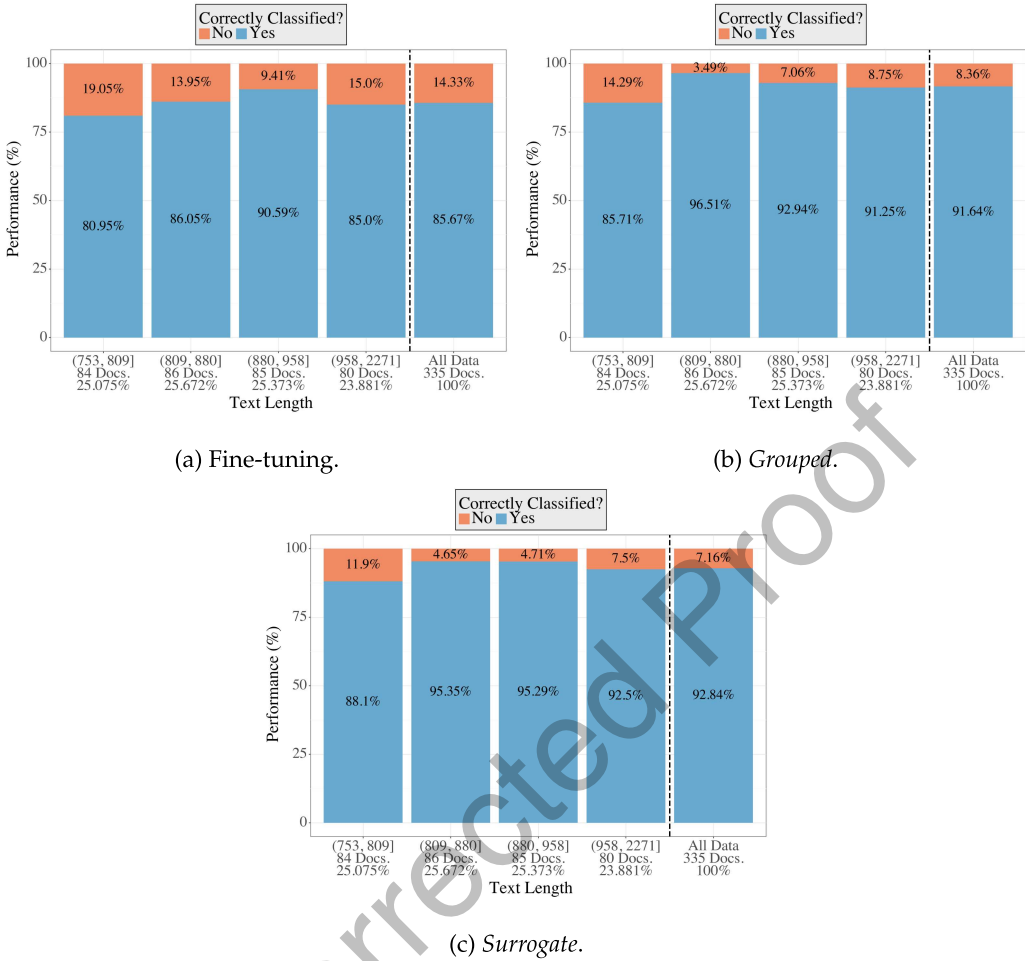


Figure G.1
EDPs over text length for fine-tuning BERT on IMDb-1k, as well as for both variants of our method. The colors represent whether or not documents are correctly classified by each approach.

accurate, independently of the document length. The relationship to the *surrogate* variant’s performance is slightly different, with the first two data bins being defined by the same performance. Similarly to what happened with fine-tuning, using our method proves more accurate overall than using sliding windows.

Lastly, we perform the same comparison but on R8 in Figure G.3. Here, a PLM-specific cutting point is added to produce the final bin (documents that are larger than BART-L’s context window) and performance represents macro F1-score. We see two interesting patterns. First, the sliding window-based model (Figure G.3a) performs better than our variants for documents that have a maximum of 39 words. Secondly, the *grouped* (Figure G.3b) and *surrogate* (Figure G.3c) variants are consistently better for documents that are too large to be fully encoded by BART-L. Interestingly, the *surrogate* variant correctly classified all these documents. However, the sample size is too small, being only composed by 4 documents. We plan to perform this analysis again on a data set composed by a wider range of text lengths and for which each bin is evenly sampled.



Figure G.2
EDPs over text length for training GATv2-based models using BART-L token graphs constructed using sliding windows as well as both variants of our method on IMDB-1k. The colors represent whether or not documents are correctly classified by each approach.

Appendix G.2 Ablation Studies

In order to understand which graph elements have the most impact on what the models learn, we perform a set of ablations over edges and nodes and compare the resulting performance against that of simply using our method, as seen in Figure G.4.

We start by assessing the impact of edge features on performance (Figure G.4a). Four different ablations are performed, namely, replacing the attentions with random values (RA) sampled from a uniform distribution between $[0, 1]$;³² replacing the final edge features with random weights (RW) sampled from another uniform distribution between $[0, 1]$; replacing all edge features with ones, i.e., unitary weights (UW); and

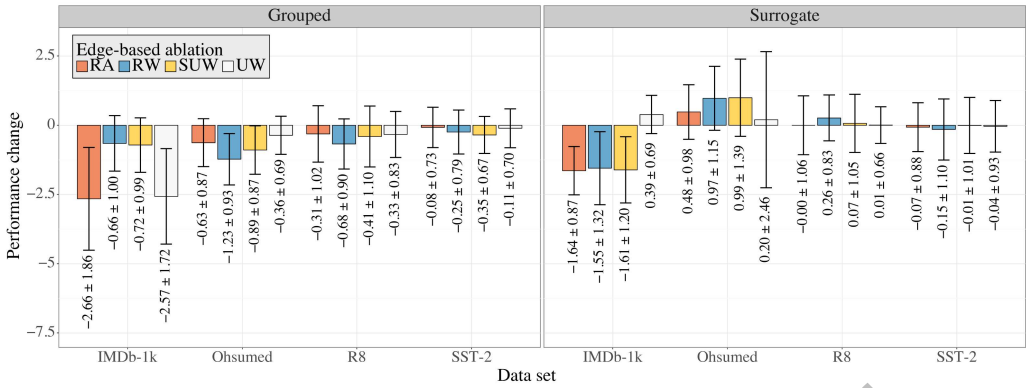
³² <https://pytorch.org/docs/stable/generated/torch.rand.html>.



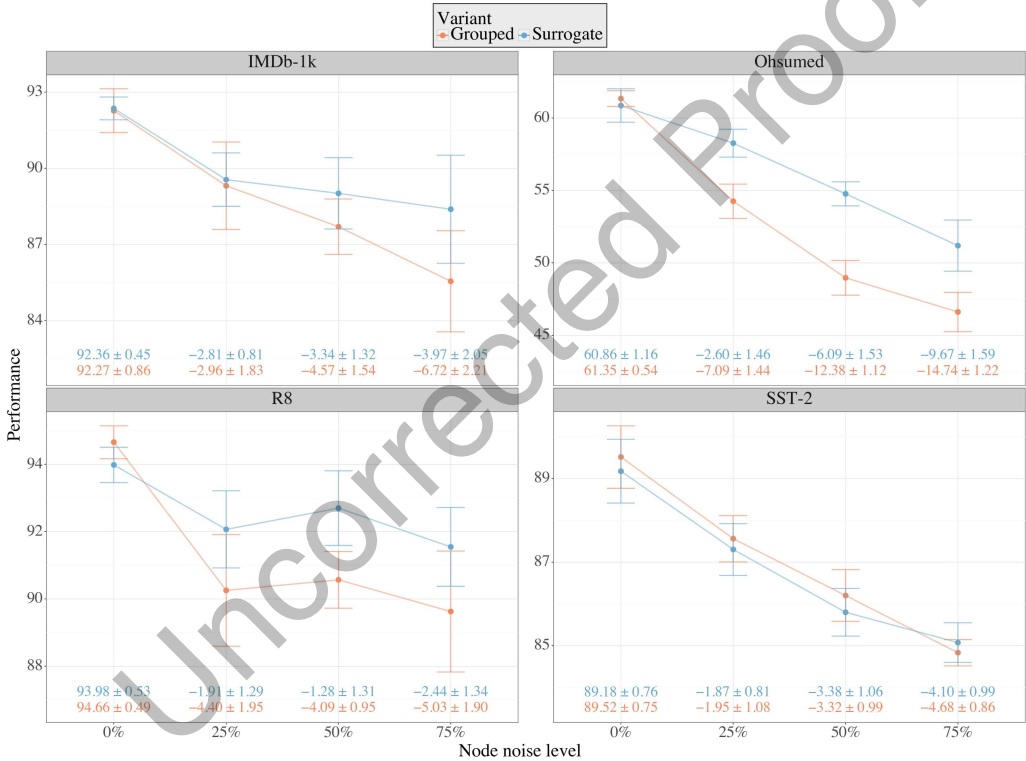
Figure G.3

EDPs over text length for training GATv2-based models using BART-L token graphs constructed using sliding windows as well as both variants of our method on R8. The colors represent whether or not documents are correctly classified by each approach.

replacing N -dimensional edge properties with a single unitary weight (SUW). Differently from RW that mainly assesses the importance of feature values, the first and the last two ablations also act as a way to study the relevance of the structure of each graph. The ablations have a consistent negative impact on the performance of the *grouped* variant, with all models performing worse than their corresponding baselines. In fact, the longer the documents in the data set, the more impacted it is by the ablations as seen by an overall trend of increasing average performance drops from SST-2 to IMDB-1k. However, some of these changes are not statistically significant (e.g., all ablations on SST-2 or UW compared to the baseline on Ohsumed), indicating that the structure of each graph, as well as its edge features may not be the most important aspects that the models learn. The results from *surrogate*-based graphs further support this suspicion, as the ablations result in an unpredicted behaviour. With the exception of RA, RW, and SUW on IMDB-1k and all ablations on SST-2, the ablations resulted in improved



(a) Edge-based ablations.



(b) Node-based ablations.

Figure G.4
Performance change (using the target metric from each benchmark data set) from applying different ablations to edge and node features using BERT as the basis for graph construction compared to not performing any ablation. Different colors represent distinct ablations in Figure G.4a and distinct method variants in Figure G.4b.

average performance. Again, some of these differences are not statistically significant (e.g., all ablations on SST-2 and R8). Nevertheless, these results suggest that the *surrogate* variant may be more affected by other graph properties (such as the contents of node embeddings) during learning than the *grouped* one. We hypothesize that this behaviour may be a direct result of some over-parametrization from the PLM (Kovaleva et al. 2019), which could be averaged out when performing the additional aggregations of the *grouped* variant.

We also evaluate how noisy node embeddings affect the quality of results in Figure G.4b. Noise is added to each node individually by randomly selecting 25%, 50%, or 75% of its features and adding uniformly sampled values from a distribution between $[0, 1)$. The resulting performance from training models using these noisy graphs is then compared to the performance of not adding any noise (i.e., our previously reported method’s performance). Regardless of the method variant, we see an overall pattern of decreasing performance with increasing noise, with all differences being statistically significant when compared to not having noise in the graphs. The bigger drops when compared to edge ablations confirm that edge features are not as important as node embeddings, independently of which variant is used. Although we hypothesized that the *surrogate* variant would be more affected by changes to node embeddings, the *grouped* one is characterized by a bigger declines for most levels of noise. We suspect that this may be due to graphs having fewer nodes compared to the *surrogate*-based graphs, resulting in less neighborhood-based smoothing during learning.

Both sets of ablations indicate that all models, regardless of method variant, rely more on node embeddings than edge features or graph structure. Nevertheless, all properties appear to have an important role in achieving the best performance. These results further support our suspicion that our graph construction pipeline may gain from more adequate pruning and edge-based feature selection, followed by extra stages in the model architecture to fully exploit the relational information in the graphs, removing some dependency from node embeddings.

Appendix G.3 Hyperparameter Importance

In order to understand how important different hyperparameters are for all experiments, we calculate a measure of importance for each BART-L-based optimization study using Optuna³³ as seen in Figure G.5. For both method variants, the threshold is the most important hyperparameter, followed by the learning rate. However, we suspect that this importance may be overestimated since Optuna’s method considers all trials, including failed ones (e.g., due to memory overflow). To confirm that threshold is in fact important, we isolated its importance in SST-2, a data set for which we have no memory problems. The threshold is the second most important hyperparameter for the *grouped* variant and fourth for the *surrogate* one with an importance estimation of ≈ 0.08 . Thus, although not as important as the boxplots may indicate, the threshold is indeed crucial to achieve the best performance. We also see the importance of adding context via padding, especially left padding in the *surrogate* variant so that each chunk has information about what comes before it. Other relevant hyperparameters are, for example, attention heads and hidden dimension for both variants, count of hidden layers for *grouped*, and dropout rate for the *surrogate* variant.

³³ https://optuna.readthedocs.io/en/stable/reference/generated/optuna.importance.get_param_importances.html.

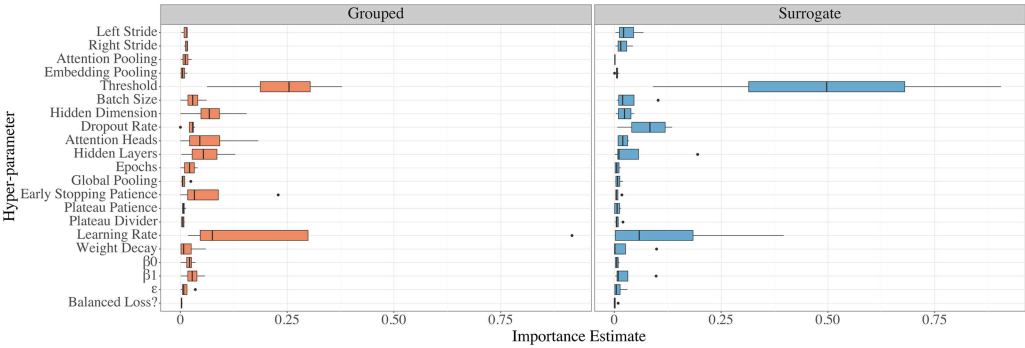


Figure G.5
Hyperparameter importance estimates from all BART-L-based Optuna studies. Each sub-plot comprises the estimates for a different method variant.

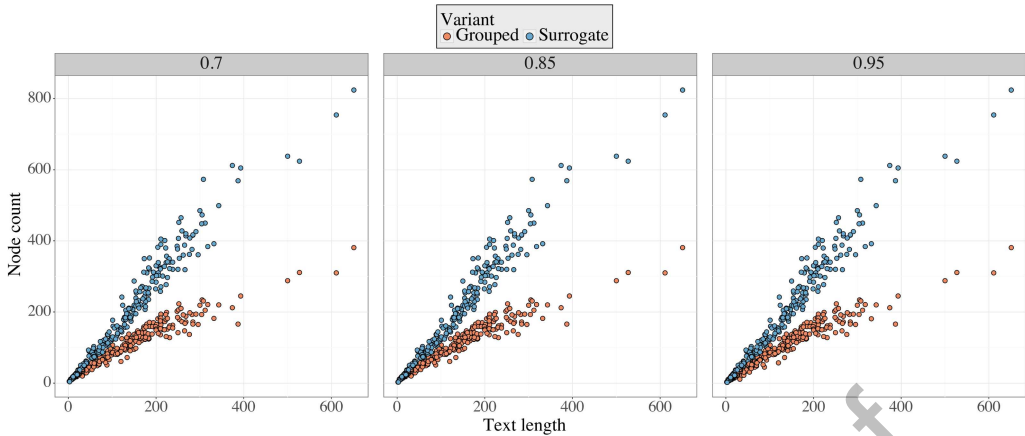
In order to try to understand whether there is an optimal combinations of values for the graph construction hyperparameters, we analyzed the performance ranks for different stride values, pooling operations, and thresholds. We could not find a combination that consistently surpasses all others, with the best values for these hyperparameters changing depending on the data set and method variant.

Appendix G.4 Memory Consumption Relationship with Text Length

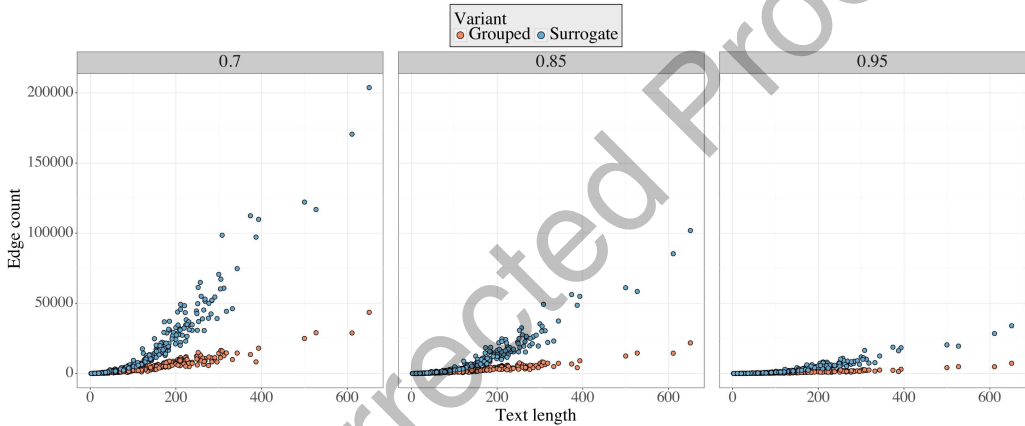
We analyze how nodes and edges relate to text length in Figure G.6. We start by randomly selecting 300 documents from the validation and testing splits of all benchmark data sets and construct their corresponding graphs. This is done using combinations of 64 and 128 left- and right-strides (when necessary), as well as three different thresholds (0.7, 0.85, and 0.95 quantiles). BART-L is used as the basis of graph construction alongside average pooling for both attentions and token embeddings. The count of nodes (Figure G.6a) follows a linear relationship with text length. The *surrogate* variant grows faster than the *grouped* one, being defined by a ratio greater than one when compared to the length estimate—a result of some words being split into smaller pieces (as previously stated, text length equals $\approx 71\%$ of token count). When using the *grouped* variant, the relationship between text length and node count can be extrapolated based on a measure of word repetition, such as TTR. For example, some of the shorter randomly selected documents have fewer repeated words than longer ones (refer to Table B.1), resulting in node counts closer to those of *surrogate*-based graphs. The thresholds we evaluate have no considerable impact on node count, being too wide to result in the drop of nodes. The number of edges in each graph appears to follow a univariate quadratic polynomial (i.e., $y = \alpha x^2$, where x is the text length and y consists of edge counts) relationship with the text length estimate, as seen in Figure G.6b. The threshold directly affects α , for which higher thresholds result in smaller α values. In other words, tightening the threshold results in fewer edges, as expected.

Appendix G.5 Time to Construct

Using the same randomly selected documents, we construct their corresponding graphs three times to account for unexpected spikes on time to construct. We aggregate the



(a) Node counts compared to text length estimates.



(b) Edge counts compared to text length estimates.

Figure G.6
Scatter plots of node and edge counts compared to text length estimates for both method variants using BART-L as construction basis. Each sub-plot consists of the values for a specific threshold (quantile), and the different colors correspond to distinct method variants.

construction times so that the threshold and stride values are averaged out, as our focus is comparing the two variants, as well as different PLMs. Figure G.7 contains two scatter plots (one for each method variant) that relate the time to construct each graph with text length estimates. We see that the *surrogate* variant is considerably slower than the *grouped* one. Although the difference may be small for shorter documents, it gets progressively larger with increasing text length. The PLM also impacts construction times noticeably. RoBERTa-L is slower than BART-L, which in turn is slower than BERT and BART. Furthermore, using *large* pre-trained models takes longer than using their corresponding simplified (*base*) models due to having more values to store and aggregate for both nodes and edges.

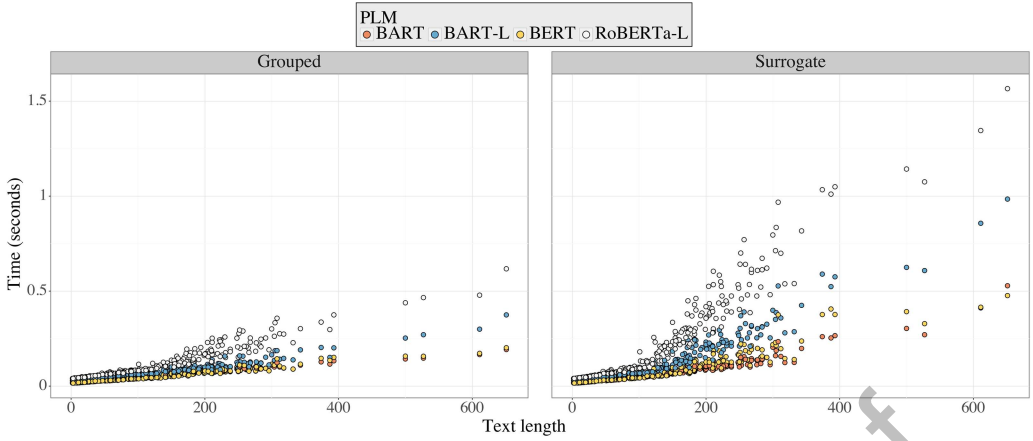


Figure G.7
Scatter plots of time to construct each graph using both method variants compared to text length. Different colors correspond to using distinct PLMs for graph construction.

Appendix G.6 Training Time and Convergence

Finally, we assess the average epoch time and overall convergence speed of models trained with graphs constructed using both our method variants, as well as with fine-tuning. We start by comparing both variants using two of the best performing PLMs (BART-L and RoBERTa-L) in Table G.1. Although the choice of the PLM appears to have a certain impact on training time, we cannot generalize the results. While RoBERTa-L is defined by the fastest training times in three of the four data sets, this behaviour appears problem- and variant-dependent. Similarly, we cannot claim that using one variant is faster than the other. For example, BART-L-based models trained with *surrogate* graphs are quicker to train than *grouped* ones in R8, but the opposite happens when using RoBERTa-L. We hypothesize that this may be caused by different hyperparameter combinations and/or specific model and data set properties that have more impact on training time and convergence than the graphs themselves.

We also compare the training times of using both our method variants with simply fine-tuning BERT in Table G.2. While fine-tuning requires fewer epochs to converge, the average epoch time of our proposal is always lower than that of fine-tuning. Total training time follows a similar trend to average epoch time, with the exception of IMDB-1k and using the *grouped* variant in Ohsumed. While the results in Ohsumed may be model-specific (e.g., more attention heads and hidden layers than the *surrogate*-based configuration), the training time in IMDB-1k is heavily linked to data-specific properties. Fine-tuning ignores much information from the documents in this data set given BERT is bound to only 512 tokens per text, which negatively impacted performance (refer to Table 1).

For data sets mostly composed of documents that fit inside the PLM’s context window, using our method is expected to be faster than fine-tuning. Contrarily, training may be longer than fine-tuning when most documents are too large for the pre-trained model to encode, which is a direct result of the extra information that needs encoding.

Table G.1
Time-related properties from training models with graphs constructed using both method variants with BART-L and RoBERTa-L as construction basis. **Bold** represents the lowest value for each pair of data set and metric.

Data Set	PLM	Variant	Average Epoch Time	Epochs to Converge	Training Time
IMDb-1k	BART-L	G. ¹	3.76 ± 0.27	13.90 ± 5.02	52.30 ± 18.16
		S. ²	1.53 ± 0.01	50.70 ± 6.25	77.58 ± 9.60
	RoBERTa-L	G. ¹	1.17 ± 0.01	20.50 ± 7.21	24.03 ± 8.48
		S. ²	6.16 ± 0.03	24.10 ± 5.82	148.51 ± 35.84
Ohsumed	BART-L	G. ¹	10.62 ± 0.24	28.30 ± 3.59	300.50 ± 38.20
		S. ²	4.17 ± 0.19	24.20 ± 6.60	100.85 ± 27.34
	RoBERTa-L	G. ¹	9.69 ± 0.04	20.80 ± 4.29	201.60 ± 41.63
		S. ²	2.70 ± 0.03	22.00 ± 5.79	59.32 ± 15.50
R8	BART-L	G. ¹	17.72 ± 0.36	103.60 ± 16.25	1835.88 ± 290.63
		S. ²	6.29 ± 0.18	26.90 ± 9.64	169.12 ± 60.38
	RoBERTa-L	G. ¹	2.64 ± 0.03	17.90 ± 6.01	47.20 ± 15.92
		S. ²	9.76 ± 0.07	37.20 ± 9.58	363.05 ± 92.88
SST-2	BART-L	G. ¹	1.27 ± 0.02	12.80 ± 6.30	16.29 ± 7.94
		S. ²	1.88 ± 0.03	15.20 ± 7.38	28.63 ± 13.83
	RoBERTa-L	G. ¹	1.91 ± 0.06	15.00 ± 3.37	28.62 ± 6.46
		S. ²	6.62 ± 0.04	35.90 ± 10.28	237.76 ± 68.05

¹Grouped. ²Surrogate.

Table G.2
Time-related properties from training models with graphs constructed using both method variants with BERT as construction basis compared to fine-tuning the PLM. **Bold** represents the lowest value for each pair of data set and metric.

Data Set	Approach	Average Epoch Time	Epochs to Converge	Training Time
IMDb-1k	Fine-tuning	13.11 ± 0.03	6.10 ± 2.51	79.95 ± 33.02
	Grouped	6.16 ± 0.39	15.70 ± 7.06	96.71 ± 42.84
	Surrogate	5.78 ± 0.05	30.20 ± 12.42	174.62 ± 71.65
Ohsumed	Fine-tuning	64.64 ± 0.04	7.40 ± 0.97	478.30 ± 62.51
	Grouped	29.20 ± 0.39	25.60 ± 2.95	747.49 ± 85.76
	Surrogate	11.72 ± 0.12	24.00 ± 3.50	281.25 ± 40.83
R8	Fine-tuning	112.74 ± 0.90	5.00 ± 2.49	563.69 ± 281.67
	Grouped	8.86 ± 0.11	22.90 ± 8.05	202.88 ± 71.26
	Surrogate	6.98 ± 0.09	43.80 ± 8.01	305.85 ± 56.14
SST-2	Fine-tuning	172.15 ± 0.23	3.90 ± 2.02	671.37 ± 348.49
	Grouped	3.97 ± 0.02	10.70 ± 7.76	42.46 ± 30.66
	Surrogate	13.52 ± 0.05	8.10 ± 4.75	109.55 ± 64.14

Appendix H: Task-based Variant and Threshold Guide

To support effective tuning, we suggest some exploratory threshold ranges and method variants depending on the task in Table H.1. Although the suggested ranges cover a wide range of scenarios, there are exceptions that demand attention. The threshold should be lower (between 0.55 and 0.85) for documents composed by few sentences and stricter for text considerably longer than the PLM’s context window, specifically between 0.90 and 0.999. However, for information extraction tasks such as extractive summarization—for which every token may contain relevant information despite each document’s length—the threshold should not exceed 0.95 to guarantee that the tokens are not prematurely filtered. Variant usage is directly tied to the task and the importance that word order has in it: The *surrogate* variant should be used for tasks in which text flow is crucial in contrast to the *grouped* alternative. For example, *grouped* graphs are not the most adequate for paraphrasing detection and summarization given how important order and individual occurrences are for the success of these tasks.

Table H.1
Suggested threshold ranges and method variants for each evaluated task.

Task	Variant	Threshold Range
Classification	Grouped/Surrogate	0.55–0.999
Regression	Grouped/Surrogate	0.55–0.999
Paraphrasing Detection	Surrogate	0.55–0.95
Summarization	Surrogate	0.55–0.95

Acknowledgments

This research is partially supported by a research grant held by Dr. Hamideh Bayrampour at the University of British Columbia. Dr. Frank Rudzicz is supported by a Canada CIFAR Chair in AI and a Killam Memorial Chair. Resources used during this research were provided, in part, by the Vector Institute for Artificial Intelligence.

References

Akiba, Takuya, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631. <https://doi.org/10.1145/3292500.3330701>

Alikaşifoğlu, Tuna, Arda Aras, and Aykut Koc. 2024. VISPool: Enhancing transformer encoders with vector visibility graph neural networks. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 2547–2556. <https://doi.org/10.18653/v1/2024.findings-acl.149>

Areosa, Inês and Luís Torgo. 2019. Explaining the performance of black box regression models. In *2019 IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, pages 110–118. <https://doi.org/10.1109/DSAA.2019.00025>

Areosa, Inês and Luís Torgo. 2020. Visual interpretation of regression error. *Expert Systems*, 37(6):e12621. <https://doi.org/10.1111/exsy.12621>

Blei, David M., Andrew Y. Ng, and Michael I. Jordan. 2002. Latent Dirichlet allocation. *Advances in Neural Information Processing Systems 14: Proceedings of the 2001 Conference*, The MIT Press. <https://doi.org/10.7551/mitpress/1120.003.0082>

Brody, Shaked, Uri Alon, and Eran Yahav. 2022. How attentive are graph attention networks? In *International Conference on Learning Representations*.

Burdisso, Sergio, Esaú Villatoro-Tello, Srikanth Madikeri, and Petr Motlicek. 2023. Node-weighted graph convolutional network for depression detection in

- transcribed clinical interviews. In *Proceedings of INTERSPEECH 2023*, pages 3617–3621. <https://doi.org/10.21437/Interspeech.2023-1923>
- Cai, Tianle, Shengjie Luo, Keyulu Xu, Di He, Tie-Yan Liu, and Liwei Wang. 2021. GraphNorm: A principled approach to accelerating graph neural network training. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139, pages 1204–1215.
- Clark, Kevin, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. 2019. What does BERT look at? An analysis of BERT’s attention. In *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 276–286. <https://doi.org/10.18653/v1/W19-4828>
- Crossley, Scott, Aron Heintz, Joon Suh Choi, Jordan Batchelor, Mehrnoush Karimi, and Agnes Malatinszky. 2021. The commonlit ease of readability (CLEAR) corpus. In *Proceedings of the 14th International Conference on Educational Data Mining (EDM)*.
- Crossley, Scott, Aron Heintz, Joon Suh Choi, Jordan Batchelor, Mehrnoush Karimi, and Agnes Malatinszky. 2023. A large-scaled corpus for assessing text readability. *Behavior Research Methods*, 55:491–507. <https://doi.org/10.3758/s13428-022-01802-x>, PubMed: 35297016
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
- Ding, Kaize, Jianling Wang, Jundong Li, Dingcheng Li, and Huan Liu. 2020. Be more with less: Hypergraph attention networks for inductive text classification. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4927–4936. <https://doi.org/10.18653/v1/2020.emnlp-main.399>
- Donabauer, Gregor and Udo Kruschwitz. 2025. Token-level graphs for short text classification. In *Advances in Information Retrieval*, pages 427–436. https://doi.org/10.1007/978-3-031-88714-7_42
- Ellson, John, Emden Gansner, Lefteris Koutsofios, Stephen C. North, and Gordon Woodhull. 2002. Graphviz—Open source graph drawing tools. In *Graph Drawing, Lecture Notes in Computer Science (LNCS)*, volume 2265, pages 483–484. https://doi.org/10.1007/3-540-45848-4_57
- Gilmer, Justin, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. 2017. Neural message passing for quantum chemistry. In *34th International Conference on Machine Learning, ICML 2017*, volume 70, pages 1263–1272.
- Grattafiori, Aaron, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, et al. 2024. The Llama 3 herd of models.
- Greene, Derek and Padraig Cunningham. 2006. Practical solutions to the problem of diagonal dominance in kernel document clustering. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 377–384. <https://doi.org/10.1145/1143844.1143892>
- Han, Xu and Zhaogong Zhang. 2022. Combining interpretability and filtering algorithm for sequence recommendation. In *2022 IEEE 8th International Conference on Computer and Communications (ICCC)*, pages 2321–2325. <https://doi.org/10.1109/ICCC56324.2022.10065875>
- Htut, Phu Mon, Jason Phang, Shikha Bordia, and Samuel R. Bowman. 2019. Do attention heads in BERT track syntactic dependencies? *arXiv*. <https://doi.org/10.48550/arXiv.1911.12246>
- Huang, Lianzhe, Dehong Ma, Sujian Li, Xiaodong Zhang, and Houfeng Wang. 2019. Text level graph neural network for text classification. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3444–3450. <https://doi.org/10.18653/v1/D19-1345>
- Jin, Yilun, Wei Yin, Haoseng Wang, and Fang He. 2024. Capturing word positions does help: A multi-element hypergraph gated attention network for document classification. *Expert Systems with Applications*, 251:124002. <https://doi.org/10.1016/j.eswa.2024.124002>
- Jin, Zhigang, Manyue Tao, Xiaofang Zhao, and Yi Hu. 2022. Social media sentiment analysis based on dependency graph and

- co-occurrence graph. *Cognitive Computation*, 14:1039–1054. <https://doi.org/10.1007/s12559-022-10004-8>
- Kim, Sehoon, Sheng Shen, David Thorsley, Amir Gholami, Woosuk Kwon, Joseph Hassoun, and Kurt Keutzer. 2022. Learned token pruning for transformers. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 784–794. <https://doi.org/10.1145/3534678.3539260>
- Kipf, Thomas N. and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*.
- Knyazev, Boris, Graham W. Taylor, and Mohamed R. Amer. 2019. Understanding attention and generalization in graph neural networks. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*.
- Kovaleva, Olga, Alexey Romanov, Anna Rogers, and Anna Rumshisky. 2019. Revealing the dark secrets of BERT. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 4365–4374. <https://doi.org/10.18653/v1/D19-1445>
- Lacasa, Lucas, Bartolo Luque, Fernando Ballesteros, Jordi Luque, and Juan Carlos Nuño. 2008. From time series to complex networks: The visibility graph. *Proceedings of the National Academy of Sciences*, 105(13):4972–4975. <https://doi.org/10.1073/pnas.0709247105>, PubMed: 18362361
- Lan, Zhenzhong, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2020. ALBERT: A lite BERT for self-supervised learning of language representations. In *International Conference on Learning Representations*.
- Lee, Junhyun, Inyeop Lee, and Jaewoo Kang. 2019. Self-attention graph pooling. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97, pages 3734–3743.
- Lewis, Mike, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880. <https://doi.org/10.18653/v1/2020.acl-main.703>
- Li, Pengyi, Xueying Fu, Juntao Chen, and Junyi Hu. 2025. CoGraphNet for enhanced text classification using word-sentence heterogeneous graph representations and improved interpretability. *Scientific Reports*, 15:356. <https://doi.org/10.1038/s41598-024-83535-9>, PubMed: 39747366
- Li, Yujia, Richard Zemel, Marc Brockschmidt, and Daniel Tarlow. 2016. Gated graph sequence neural networks. In *4th International Conference on Learning Representations, ICLR 2016 – Conference Track Proceedings*.
- Lin, Yuxiao, Yuxian Meng, Xiaofei Sun, Qinghong Han, Kun Kuang, Jiwei Li, and Fei Wu. 2021. BertGCN: Transductive text classification by combining GNN and BERT. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 1456–1462. <https://doi.org/10.18653/v1/2021.findings-acl.126>
- Linmei, Hu, Tianchi Yang, Chuan Shi, Houye Ji, and Xiaoli Li. 2019. Heterogeneous graph attention networks for semi-supervised short text classification. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 4821–4830. <https://doi.org/10.18653/v1/D19-1488>
- Liu, Bing, Xiaoli Li, Wee Sun Lee, and Philip S. Yu. 2004. Text classification by labeling words. In *Proceedings of the National Conference on Artificial Intelligence*.
- Liu, Ninghao, Qizhang Feng, and Xia Hu. 2022. Interpretability in graph neural networks. *Graph Neural Networks: Foundations, Frontiers, and Applications*, pages 121–147. Springer. https://doi.org/10.1007/978-981-16-6054-2_7
- Liu, Yinhan, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv*. <https://doi.org/10.48550/arXiv.1907.11692>
- Loshchilov, Ilya and Frank Hutter. 2017. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*.
- Maas, Andrew L., Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. 2011. Learning word

- vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 142–150.
- Michel, Paul, Omer Levy, and Graham Neubig. 2019. Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems*, volume 32, Curran Associates, Inc.
- Mikolov, Tomas, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems*, volume 26, Curran Associates, Inc.
- Narang, Sharan, Hyung Won Chung, Yi Tay, Liam Fedus, Thibault Fevry, Michaela Matena, Karishma Malkan, Noah Fiedel, Noam Shazeer, Zhenzhong Lan, Yanqi Zhou, Wei Li, Nan Ding, Jake Marcus, Adam Roberts, and Colin Raffel. 2021. Do transformer modifications transfer across implementations and applications? In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 5758–5773. <https://doi.org/10.18653/v1/2021.emnlp-main.465>
- Nikolentzos, Giannis, Antoine Tixier, and Michalis Vazirgiannis. 2020. Message passing attention networks for document understanding. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34:8544–8551. <https://doi.org/10.1609/aaai.v34i05.6376>
- Pang, Bo and Lillian Lee. 2005. Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales. In *Proceedings of the 43rd Annual Meeting of the Association for Computational Linguistics (ACL-05)*, pages 115–124. <https://doi.org/10.3115/1219840.1219855>
- Pedregosa, Fabian, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.
- Pennington, Jeffrey, Richard Socher, and Christopher Manning. 2014. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543. <https://doi.org/10.3115/v1/D14-1162>
- Radford, Alec, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *arXiv:2406.14491*.
- Ravishankar, Vinit, Artur Kulmizev, Mostafa Abdou, Anders Søgaard, and Joakim Nivre. 2021. Attention can reflect syntactic structure (if you let it). In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 3031–3045. <https://doi.org/10.18653/v1/2021.eacl-main.264>
- Ren, Weikai Ren and Ningde Jin. 2019. Vector visibility graph from multivariate time series: A new method for characterizing nonlinear dynamic behavior in two-phase flow. *Nonlinear Dynamics*, 97(4):2547–2556. <https://doi.org/10.1007/s11071-019-05147-7>
- Sanh, Victor, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv*. <https://doi.org/10.48550/arXiv.1910.01108>
- Socher, Richard, Alex Perelygin, Jean Y. Wu, Jason Chuang, Christopher D. Manning, Andrew Y. Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642. <https://doi.org/10.18653/v1/D13-1170>
- Taha, Kamal, Paul D. Yoo, Chan Yeun, Dirar Homouz, and Aya Taha. 2024. A comprehensive survey of text classification techniques and their research applications: Observational and experimental insights. *Computer Science Review*, 54:100664. <https://doi.org/10.1016/j.cosrev.2024.100664>
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, Curran Associates, Inc.
- Veličković, Petar, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2018. Graph attention networks. In *International Conference on Learning Representations*.
- Vig, Jesse. 2019. A multiscale visualization of attention in the transformer model. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*:

- System Demonstrations*, pages 37–42.
<https://doi.org/10.18653/v1/P19-3007>
- Voita, Elena, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov. 2019. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 5797–5808.
<https://doi.org/10.18653/v1/P19-1580>
- Wang, Tianming and Xiaojun Wan. 2019. Hierarchical attention networks for sentence ordering. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(1):7184–7191. <https://doi.org/10.1609/aaai.v33i01.33017184>
- Wang, Youwei, Lizhou Feng, Ao Liu, Weiqi Wang, and Yudong Hou. 2024. Dual BiGRU-CNN-based sentiment classification method combining global and local attention. *The Journal of Supercomputing*, 80:2799–2837. <https://doi.org/10.1007/s11227-023-05558-9>
- Yang, Yiping and Xiaohui Cui. 2021. BERT-enhanced text graph neural network for classification. *Entropy*, 23:1536.
<https://doi.org/10.3390/e23111536>
 PubMed: 34828233
- Yang, Zhilin, Zihang Dai, Yiming Yang, Jaime Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. 2019. XLNet: Generalized autoregressive pretraining for language understanding. In *Advances in Neural Information Processing Systems*, volume 32, Curran Associates, Inc.
- Yang, Zichao, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. 2016. Hierarchical attention networks for document classification. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1480–1489.
<https://doi.org/10.18653/v1/N16-1174>
- Yao, Liang, Chengsheng Mao, and Yuan Luo. 2019. Graph convolutional networks for text classification. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33:7370–7377. <https://doi.org/10.1609/aaai.v33i01.33017370>
- Zhang, Junjun, Zhengyan Cui, Hyun Jun Park, and Giseop Noh. 2022. BHGAttn: A feature-enhanced hierarchical graph attention network for sentiment analysis. *Entropy*, 24:1691. <https://doi.org/10.3390/e24111691> PubMed: 36421546
- Zhang, Yuan, Jason Baldridge, and Luheng He. 2019. PAWS: Paraphrase adversaries from word scrambling. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, volume 1 (Long and Short Papers), pages 1298–1308.
- Zhang, Yufeng, Xueli Yu, Zeyu Cui, Shu Wu, Zhongzhen Wen, and Liang Wang. 2020. Every document owns its structure: Inductive text classification via graph neural networks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 334–339.
<https://doi.org/10.18653/v1/2020.acl-main.31>
- Zhao, Yuxia, Mahpirat Mamat, Alimjan Aysa, and Kurban Ubul. 2024. A dynamic graph structural framework for implicit sentiment identification based on complementary semantic and structural information. *Scientific Reports*, 14:16563.
<https://doi.org/10.1038/s41598-024-62269-8>, PubMed: 39019898